



UNIVERSIDAD DE LAS CIENCIAS INFORMÁTICAS
FACULTAD DE TECNOLOGÍAS INTERACTIVAS

**SERVICIO BACKEND BASADO EN EL PARADIGMA DE LÍNEAS DE PRODUCTOS
DE SOFTWARE Y MODELOS DE CARACTERÍSTICAS PARA LA GESTIÓN
SISTEMÁTICA DE LA VARIABILIDAD CURRICULAR EN INSTITUCIONES DE
EDUCACIÓN SUPERIOR**

Trabajo de diploma para optar por el título de Ingeniero en Ciencias Informáticas

Autor: Jose Luis Echemendia López

Tutores: MSc. Yadira Ramírez Rodríguez

Ing. Liany Sobrino Miranda

La Habana, 2026

What is well set on your mind will be for sure well set on your lips
José Martí

Borrador

To my wife, my endless love

I am greatly indebted to professor John...

Declaración de auditoría

Declaramos ser autores de la presente tesis y reconocemos a la Universidad de las Ciencias Informáticas los derechos patrimoniales sobre esta, con carácter exclusivo.

Para que así conste firmamos la presente a los ____ días del mes de _____ del año _____.

Jose Luis Echemendia López
Autor

MSc. Yadira Ramírez Rodríguez
Tutor

Ing. Liany Sobrino Miranda
Tutor

En el contexto de la rápida evolución tecnológica y las transformaciones sociales del siglo XXI, la Educación Superior enfrenta el desafío de gestionar currículos flexibles que respondan a demandas cambiantes. Sin embargo, esta tarea se ve constantemente superada por el vertiginoso ritmo del cambio tecnológico, que amenaza con dejar obsoleto cualquier modelo educativo, por más innovador que sea en su concepción, antes de que pueda ser plenamente implementado y consolidado. Además, el mantenimiento de la variabilidad curricular se realiza actualmente mediante prácticas artesanales basadas en documentos estáticos y procesos manuales, un enfoque que resulta insostenible por su lentitud, propensión a errores e incapacidad para garantizar la trazabilidad y coherencia de los planes de estudio.

Esta investigación aborda este problema aplicando los principios de la Ingeniería de Líneas de Productos de Software (SPL) y los Modelos de Características (*Feature Models* - FM) al dominio de la planificación curricular. El trabajo se centra en el diseño, implementación y validación de un *backend* que permita a los gestores académicos modelar currículos dando lugar a la variabilidad curricular mediante FMs, con el objetivo de generar "Líneas de Productos Educativos". El sistema cuenta con una API RESTful que sirve como interfaz para las operaciones de creación, análisis, validación y configuración de FMs, permitiendo representar características y relaciones de obligatoriedad, optatividad y prerrequisitos como restricciones formales.

El resultado principal es un servicio backend especializado que no solo soporta el modelado y la persistencia de FMs adaptados al ámbito educativo, sino que también proporciona la base computacional para automatizar la generación y validación de planes de estudio diversos y consistentes a partir de un tronco común. Este desarrollo sienta las bases técnicas para una nueva generación de herramientas de gestión académica, demostrando la viabilidad de transferir técnicas avanzadas de ingeniería de software al dominio educativo.

Palabras clave: Líneas de Productos Educativos, Líneas de Productos de Software, Modelos Característicos, variabilidad curricular.

Lista de tareas pendientes	xiii
Introducción	1
1 Fundamentación Teórica	5
1.1 Conceptos asociados al tema	5
1.1.1 Ingeniería de Líneas de Productos de Software (SPLE)	5
1.1.2 Modelos de Características (Feature Models)	7
1.1.3 Lenguaje Universal de Variabilidad (UVL)	14
1.1.4 Configuraciones y configurador curricular	17
1.1.5 Gestión y flexibilidad curricular. Línea de Productos Educativos	18
1.2 Fortalezas de los Modelos de Característica aplicado en la gestión curricular	19
1.2.1 Analogía formal curricular	20
1.2.2 Beneficios de las Líneas de Productos Educativos	26
1.3 Análisis del mercado	26
1.3.1 Panorama General del Mercado	27
1.3.2 Análisis de Soluciones Existentes	27
1.3.3 Oportunidades de Mercado Identificadas	29
1.4 Herramientas y tecnologías	31
1.5 Metodología de desarrollo del software	31
1.5.1 Metodologías de desarrollo de software tradicionales	31
1.5.2 Metodologías de desarrollo de software ágiles	32
1.5.3 Elección de la metodología	33
1.5.4 Programación Extrema	35
1.6 Conclusiones del capítulo	36
2 Propuesta de solución	37
2.1 Sección de prueba	37
2.2 Ejemplos de código fuente	38

3 Ejemplo de tabla grande	44
3.1 Tabla aleatoria	44
3.2 Tablas de ingeniería	45
4 Resultados	50
Conclusiones	52
Recomendaciones	53
Referencias bibliográficas	54
Apéndices	58
A Proyectos y Avaes	59
B Otro reporte	60
B.1 Otra sección de muestra	60
B.2 Otro ensamble Industrial	60

Índice de figuras

1.1	Símbolos gráficos para representar las relaciones estructurales.	10
1.2	Representación gráfica del FM del modelo Sandwich especificado en UVL. Fuente: elaboración propia.	16
1.3	Representación simplificada #1 del Modelo de Características (FM) de la asignatura Estructura de Datos I. Fuente: elaboración propia.	23
1.4	Representación simplificada #2 del FM de la asignatura Estructura de Datos I. Fuente: elaboración propia.	24

Índice de cuadros

1.1	Notación semántica de las relaciones entre características de los FM	8
1.2	Analogía de conceptos de Línea de Productos de Software (SPL) en el Dominio Educativo. Fuente: Elaboración propia.	20
1.3	Comparación, Metodología Ágil VS Metodología Tradicional	33
1.4	Comparación de la Metodología Ágil y la Metodología Tradicional con respecto al proyecto	34
1.5	Comparación de Metodología Ágiles. Fuente: Elaboración propia	34
3.1	Feasible triples for highly variable Grid, MLMMH.	44
3.1	continued from previous page	45
3.2	Historia de usuario # 1	45
3.3	Tarjeta CRC # 1	46
3.4	Tarjeta CRC # 2	46
3.5	Estimación de esfuerzo por historia de usuario	46
3.6	Estimación de esfuerzo por historia de usuario	47
3.7	Plan de duración de las iteraciones	47
3.8	Tarea de ingeniería # 1	47
3.9	Tarea de desarrollo # 1	47
3.10	Tarea de desarrollo # 2	48
3.11	Tarea de ingeniería # 2	48
3.12	Prueba de aceptación # 1	48
3.13	Prueba de aceptación # 2	48

Lista de algoritmos

1	Estrategia de BellmanKalaba	39
---	---------------------------------------	----

Lista de códigos fuentes

1.1	Ejemplo de modelo de variabilidad en Lenguaje de Variabilidad Universal (UVL)	15
1.2	Ejemplo de modelo de variabilidad en UVL	21
2.1	Búsqueda de máximo	39
2.2	Ejemplo de código en Java	40
2.3	Ejemplo de código en PHP	40
2.4	Ejemplo de código en PHP	41
2.5	Ejemplo de código en Python	41
2.6	Ejemplo de código en htmlcssjs	42
2.7	Ejemplo de código en htmlcssjs	42

Lista de tareas pendientes

El panorama de la Educación Superior en el siglo XXI está inmerso en una dinámica de cambio sin precedentes, impulsada por rápidos avances tecnológicos y profundas transformaciones sociales. La aceleración tecnológica y las fluctuantes demandas del mercado laboral exigen que las instituciones académicas migren de currículos rígidos hacia planes de estudio dinámicos y flexibles (UNESCO, 2022). Esta necesidad de flexibilidad introduce un desafío crítico que se sitúa en el núcleo de la gestión universitaria moderna: la gestión de la variabilidad curricular.

Para abordar este desafío, es necesario comprender sus dos dimensiones fundamentales. Por un lado, la innovación educativa, la cual se refiere a la introducción de nuevas ideas, enfoques, metodologías y tecnologías en el ámbito educativo; busca mejorar la calidad de la enseñanza y el aprendizaje, fomentando la creatividad, el pensamiento crítico y la resolución de problemas en los estudiantes (Gómez, 2023). Por otro lado, la gestión curricular, que implica el diseño, desarrollo e implementación de planes de estudio y programas educativos. Esta se ocupa de organizar y estructurar los contenidos, las habilidades y las competencias que se enseñan, así como la evaluación y mejora continua de los programas para asegurar su relevancia y efectividad (ibíd.). Ambas son aspectos fundamentales para el desarrollo de sistemas educativos actuales.

Para que la innovación educativa y la gestión curricular puedan responder efectivamente a las demandas de un entorno cambiante, la literatura especializada señala la necesidad de adoptar un enfoque de gestión por procesos y mejora continua en las instituciones de Educación Superior (Del Águila et al., 2014). Este enfoque, inspirado en modelos de gestión de calidad como la norma ISO 9001, implica superar la visión fragmentada de la administración académica para concebir el currículo como un sistema integrado de procesos interrelacionados que deben ser evaluados y optimizados permanentemente (Soto Grant, 2022). La gestión por procesos permite identificar con precisión las actividades clave —desde el diseño de planes de estudio hasta la evaluación del aprendizaje—, establecer responsables, y documentar procedimientos para garantizar la trazabilidad y la coherencia institucional (Veiga Méndez, 2024). Investigaciones aplicadas en universidades latinoamericanas demuestran que la implementación de este enfoque contribuye a reducir los “cuellos de botella” administrativos, optimizar el uso de recursos y alinear las operaciones cotidianas con los objetivos estratégicos de la institución (Gutama; Ortiz González; Moscoso Berna y Ayabaca Landi, 2025). Bajo esta óptica, la gestión curricular deja de ser una tarea puramente administrativa para convertirse en un ejercicio estratégico de diseño y rediseño continuo, fundamentado en ciclos de planificación, ejecución, verificación y actuación.

Este enfoque estratégico de gestión por procesos cobra especial relevancia al comprender que la innova-

ción educativa y la gestión curricular no son dimensiones independientes, sino que se encuentran estrechamente interrelacionadas. La introducción de enfoques innovadores en la enseñanza y el aprendizaje requiere una gestión curricular flexible y adaptable, que permita revisar y actualizar constantemente los planes de estudio para integrar nuevas metodologías, tecnologías y recursos educativos. Un ejemplo claro es la integración de las [Tecnologías de la Información y Comunicación \(TIC\)](#) en el aula, cuyo máximo beneficio se logra cuando la gestión curricular planifica su uso estratégico (Gómez, 2023). Esta interdependencia se ha vuelto crítica con la proliferación de nuevas modalidades educativas (presenciales, híbridas, en línea, [Curso en Línea Masivo y Abierto \(MOOC\)](#), *micro-learning*) y enfoques pedagógicos (aprendizaje basado en proyectos, *flipped classroom*, gamificación), lo que ha creado un panorama de extrema variabilidad instruccional (Bates, 2019).

A pesar de esta creciente complejidad, la definición y el mantenimiento de la variabilidad curricular en las instituciones de Educación Superior se gestionan actualmente mediante prácticas obsoletas. La dependencia de documentos estáticos, hojas de cálculo y procesos manuales constituye un enfoque fragmentado que resulta costoso, lento y altamente propenso a errores. Esta gestión artesanal dificulta la actualización oportuna de la información, genera inconsistencias entre versiones y complica la trazabilidad de los cambios. En la práctica, cada nuevo curso o recurso educativo se desarrolla prácticamente desde cero, desaprovechando la oportunidad de reutilizar componentes comunes y construir conocimiento de manera incremental sobre bases previamente validadas.

Las consecuencias de este paradigma obsoleto son profundas y de gran alcance. La falta de un enfoque sistemático provoca la divulgación de información desactualizada, compromete la coherencia institucional y puede llevar al diseño de rutas formativas incoherentes que afectan la progresión del estudiante. Esto incrementa la carga de trabajo administrativo y docente, limita la visibilidad global del currículo y tiene un impacto directo en la satisfacción del estudiante y en los procesos de acreditación. Irónicamente, la misma tecnología que ha posibilitado la explosión de modalidades y enfoques pedagógicos carece, en la actualidad, de los mecanismos sistemáticos para gestionar la complejidad que ella misma ha generado. Existe una batalla contrarreloj, donde la velocidad de la innovación tecnológica supera la capacidad de reacción de los sistemas educativos tradicionales, volviendo potencialmente obsoleta cualquier renovación curricular incluso antes de su implementación (Sánchez; Gutiérrez y Cabrera, 2020). Esta situación deja de manifiesto que el paradigma artesanal de diseño curricular ha llegado a su límite.

De este análisis, se desprende la siguiente interrogante que define el **problema de investigación**: ¿Cómo gestionar sistemáticamente la variabilidad curricular en instituciones de Educación Superior, de manera que se superen las limitaciones de los enfoques basados en documentación manual y se garantice la coherencia, trazabilidad y sostenibilidad de los planes de estudio? Tomando como **objeto de estudio**: la gestión de la variabilidad curricular en instituciones de Educación Superior. Enmarcado en el **campo de acción**: La aplicación del paradigma de Líneas de Productos de Software para la modelación y gestión de la variabilidad curricular. Con el fin de solucionar la problemática planteada, se define como **objetivo general**: desarrollar un servicio *backend* basado en el paradigma de [SPL](#) y [FM](#) para la gestión sistemática de la variabilidad

curricular en instituciones de Educación Superior.

Para dar cumplimiento al objetivo planteado se proponen las siguientes **tareas investigativas**:

1. **Estudio** del estado del arte en la [Ingeniería de Líneas de Productos de Software \(SPLE\)](#), [FM](#) y su aplicación al dominio educativo.
2. **Análisis** de diferentes soluciones homólogas para la identificación de características generales y tendencias en este tipo de aplicaciones.
3. **Selección** de las herramientas, tecnologías y metodología a emplear, para garantizar una excelente y eficiente experiencia de desarrollo y alineado con los objetivos del proyecto.
4. **Identificación** de los requisitos funcionales y no funcionales necesarios para un sistema capaz de gestionar la variabilidad de Líneas de Productos Educativos.
5. **Diseño** de la arquitectura del servicio *backend*, definiendo los componentes principales y el modelo de datos para representar [FM](#) en el contexto educativo.
6. **Implementación** del servicio *backend* para materializar el sistema que dará respuesta a la problemática planteada, asegurando la creación, análisis, validación y configuración de [FM](#).
7. **Validación** del servicio desarrollado mediante casos de uso representativos en el ámbito educativo, evaluando que cumpla con el funcionamiento esperado.

Para darle solución a los objetivos trazados en las tareas de investigación se emplearon los siguientes **métodos científicos**:

- **Métodos teóricos:**

- **Modelación:** se utilizó para la elaboración de los diagramas del lenguaje de modelado y en la representación de las características de la solución.
- **Histórico-Lógico:** Se empleó para comprender la evolución de las técnicas de gestión de variabilidad en [SPL](#), desde sus orígenes en ingeniería de software hasta su aplicación potencial en dominios educativos, analizando su desarrollo histórico y lógica interna.
- **Analítico-sintético:** Utilizado en el examen crítico de la bibliografía especializada sobre [SPL](#), [FM](#) y sistemas educativos, permitiendo sintetizar un marco conceptual adaptado al dominio educativo.

- **Métodos empíricos:**

- **Observación:** Mediante este método se analizaron herramientas existentes de gestión [SPL](#) (como *pure::variants*, *FeatureIDE*) para identificar limitaciones técnicas y requisitos necesarios para el contexto educativo específico.
- **Entrevista:** Se entrevista especialistas responsables de diseños curriculares que compran aspectos técnicos para identificar los desafíos existentes en la flexibilidad y variabilidad de los currículos.
- **Experimentación:** Mediante la implementación de prototipos funcionales para validar diferentes enfoques arquitectónicos y algoritmos de validación.

Estructura de la investigación:

Capítulo 1: Fundamentación teórica, capítulo encargado de definir los elementos teóricos necesarios para el desarrollo de la investigación y los principales conceptos que se emplearán durante todo el trabajo. Se realiza un análisis de las soluciones similares y se selecciona la metodología de desarrollo de software y las herramientas y tecnologías a utilizar.

Capítulo 2: Características y diseño del sistema, capítulo poseedor de los diferentes requisitos funcionales y no funcionales de la aplicación a desarrollar, así como de la modelación de la solución mediante los artefactos planteados por la metodología de desarrollo a utilizar.

Capítulo 3: Implementación y pruebas, capítulo que contiene los estándares de codificación utilizados en la implementación, así como las diferentes pruebas a las que será sometida la aplicación una vez terminada para verificar su correcto funcionamiento.

Fundamentación Teórica

La base teórica funciona como el almacén conceptual sobre el cual se erige todo trabajo investigativo. Este capítulo expone los conceptos asociados al tema, los fundamentos teóricos existentes y el estado del arte que respaldan el estudio, proporcionando un sistema de referencia que orienta la comprensión e interpretación del fenómeno abordado. En tal sentido, se establece el marco conceptual necesario para contextualizar y dar sentido a la propuesta de solución que se desarrolla en los capítulos subsiguientes.

Además, se efectúa una comparación entre soluciones homólogas existentes para determinar características predominantes y conocer las tendencias en este tipo de sistemas. También se realiza un análisis de las diferentes herramientas y tecnologías para dar a conocer el conjunto de utensilios que serán protagonistas en el desarrollo de la propuesta de solución, así como la metodología de desarrollo de software que guiará todo el proceso práctico.

1.1. Conceptos asociados al tema

En este epígrafe se desglosan los conceptos esenciales que convergen en el núcleo del estudio. La clarificación de estos términos no solo delimita el campo de acción, sino que también proporciona el lenguaje común y la base teórica necesaria para el diseño de una solución coherente y efectiva. Esta precisión conceptual resulta fundamental, pues sienta las bases sobre las cuales se articularán posteriormente las decisiones de diseño e implementación de la propuesta.

1.1.1. Ingeniería de Líneas de Productos de Software (SPLE)

La **SPLE** es una estrategia de ingeniería de software que busca producir familias de sistemas de software similares a partir de un conjunto común de activos centrales mediante un proceso gestionado de desarrollo y gestión de la variabilidad. El objetivo principal es lograr la reutilización a gran escala, lo que se traduce en una reducción de costos, menor tiempo de comercialización y mayor calidad de los productos (SG Buzz, [s.f.]). El concepto de **SPL** surgió a finales de los años 90 como respuesta directa a la crisis de productividad

que enfrentaba la industria del software a pesar de las técnicas de reutilización orientadas a objetos. La [SPLE](#) formaliza una práctica que ya existía intuitivamente en muchas empresas: construir un software base (plataforma) para generar rápidamente múltiples productos similares con variaciones controladas (AcademiaLab, [\[s.f.\]](#))

Para comprender cabalmente el paradigma de [SPLE](#), es necesario examinar los conceptos fundamentales que constituyen su base teórica y que permiten articular la gestión de la variabilidad como eje central de la estrategia. Estos conceptos son:

1. **Línea de Productos:** Es la familia completa de sistemas de software que pueden ser generados. Representa el espacio de posibilidades definido por el conjunto de activos.
2. **Activos Centrales (Core Assets):** Son los componentes fundamentales (código, arquitectura, modelos de diseño, documentación, planes de prueba) diseñados específicamente para ser compartidos y reutilizados en toda la línea. La inversión inicial en estos activos es clave para el éxito de la estrategia.
3. **Variabilidad:** Es la propiedad esencial de una [SPL](#). Define la capacidad de la línea para ser adaptada, personalizada o configurada para generar productos específicos que satisfagan las necesidades particulares de un cliente o escenario. La variabilidad es, en esencia, el conjunto de diferencias controladas entre los productos posibles.
4. **Puntos de Variabilidad (Variability Points):** Son las ubicaciones explícitas dentro de los Activos Centrales donde se deben tomar decisiones para configurar un producto. Estos puntos actúan como interruptores.º .opciones"que guían el proceso de personalización.

La motivación detrás de la [SPL](#) es fundamentalmente económica y técnica, pues permite el ahorro de costos, ya que al compartir la gran mayoría de los activos (entre el 70 % y 90 % del sistema), se evita la reimplementación del mismo código y diseño para cada nuevo producto, además, provee de una reducción del tiempo de comercialización (*Time-to-Market*) pues la creación de un nuevo producto se reduce de un ciclo completo de desarrollo a un simple proceso de configuración y ensamblaje de activos preexistentes y permite una mejora de la calidad, pues como los activos centrales, al ser reutilizados y probados en múltiples productos, alcanzan un nivel de madurez y robustez superior al que podría lograrse en un desarrollo de producto único (Ingeniería de Software, [\[s.f.\]](#)).

La [SPL](#) opera a través de dos procesos interdependientes que deben gestionarse de manera continua, siendo el primero de estos la Ingeniería del Dominio (*Domain Engineering*), proceso hacia arriba que se enfoca en la creación y el mantenimiento de los Activos Centrales, cuya tarea principal es el modelado de la variabilidad para comprender y documentar todas las posibles configuraciones que la línea puede soportar y que constituye la base teórica y formal de la línea, y un segundo, la Ingeniería de la Aplicación (*Application Engineering*), el cual es el proceso hacia abajo que utiliza los Activos Centrales para configurar y generar productos específicos, que implica tomar las decisiones de variabilidad definidas en el modelo (seleccionar las opciones) para producir una instancia concreta de la línea de productos. La herramienta principal y estándar de facto para formalizar y gestionar la variabilidad durante la Ingeniería del Dominio es el [FM](#).

Este modelo establece el puente formal entre las opciones de la línea de productos y la capacidad de generar una instancia válida, lo cual es el principio que se busca trasladar al diseño curricular.

La búsqueda de la variabilidad curricular ha llevado a la exploración de paradigmas de ingeniería de software. El concepto de **SPL** ofrece un enfoque estructurado y probado para gestionar familias de productos a partir de un conjunto común de activos (Apel; Batory; Kästner y Saake, 2013). Cuando este paradigma se aplica al dominio educativo, surge el concepto de "Líneas de Productos Educativos", donde un tronco curricular común puede derivar en múltiples planes de estudio especializados mediante la gestión explícita de su variabilidad.

Evolución y Mantenimiento de Líneas de Productos

Las **SPL** no son entidades estáticas; evolucionan con el tiempo para adaptarse a nuevos requisitos, tecnologías y mercados. La evolución de **SPL** se refiere al proceso de modificar los activos centrales y los modelos de variabilidad para incorporar cambios, corregir defectos o mejorar la línea (*ibíd.*).

Los principales desafíos en la evolución de **SPL** incluyen:

- Trazabilidad de cambios: Mantener registro de qué cambios se realizaron, por qué y cuándo, así como su impacto en los productos derivados.
- Impacto en productos existentes: Determinar cómo las modificaciones en la línea afectan a los productos ya desplegados y si es necesario migrarlos.
- Consistencia del modelo: Garantizar que después de los cambios, el **FM** siga siendo consistente y represente fielmente la variabilidad deseada.
- Gestión de versiones: Coordinar diferentes versiones de la línea que puedan coexistir para dar soporte a productos en distintos estados de madurez.

En el dominio educativo, la evolución curricular es una realidad constante: nuevos planes de estudio, actualización de contenidos, cambios en normativas, etc. Un sistema de gestión de variabilidad curricular debe contemplar mecanismos para:

- Registrar el histórico de modificaciones curriculares.
- Evaluar el impacto de los cambios en los planes de estudio existentes.
- Gestionar múltiples versiones de un mismo plan (por ejemplo, planes de estudio con diferentes años de implementación).
- Asegurar la trazabilidad entre las versiones del modelo y los productos generados.

1.1.2. Modelos de Características (Feature Models)

Es necesario contar con un modelo que represente de manera explícita las variaciones y opciones disponibles en la línea de productos. Este modelo debe ser capaz de capturar la complejidad de las relaciones entre las características y sus variaciones, así como las restricciones que pueden existir entre ellas (Pohl; Böckle

y Linden, 2005). La representación gráfica de estas relaciones es fundamental para facilitar la comprensión y el análisis de la variabilidad en la línea de productos.

El Modelo de Características (*Feature Model* - FM) es la técnica de modelado central en SPLE. Su formalismo es indispensable para pasar de la noción abstracta de variabilidad a una representación estructurada y analizable (PTC, 2025). Los FM fueron formalmente introducidos por Kang; Cohen; Hess; Novak y Peterson (1990) como parte de la metodología *Análisis de Dominio Orientado a Características* (FODA), un enfoque pionero para la ingeniería de dominios. Desde su concepción, un FM ha sido reconocido como el artefacto primario para la captura de la variabilidad funcional de un sistema. En esencia, un FM es un modelo de dominio que no solo documenta las posibles funcionalidades, sino que también define las reglas de composición entre ellas (GeeksforGeeks, [s.f.]).

Formalmente, un FM se define como un árbol de decisiones jerárquico donde cada nodo es una característica (*feature*). Una característica se entiende como una unidad de funcionalidad, un requisito, o un aspecto distintivo de un sistema, tal como es percibido por el usuario final o el analista del dominio.

La estructura del FM es un árbol de composición, donde:

- La característica raíz representa el sistema o la línea de producto completa (ej., la carrera de Ingeniería Informática).
- Las características descendientes detallan la composición de sus características padre hasta alcanzar las características hoja, que son las unidades más pequeñas y configurables.

Relaciones estructurales

La potencia del FM reside en su notación gráfica concisa, la cual utiliza la posición y símbolos específicos para codificar las restricciones de configuración. Las relaciones entre una característica padre (letra P) y sus hijos (letra C) definen las decisiones de inclusión o exclusión; véase la Tabla 1.1 (D. Benavides; S. Segura y A. Ruiz-Cortés, 2010; David Benavides; Sergio Segura y Antonio Ruiz-Cortés, 2010; Kang; Cohen; Hess; Novak y Peterson, 1990; Pohl; Böckle y Linden, 2005; Wang; Zhao y Hu, 2007).

Cuadro 1.1. Notación semántica de las relaciones entre características de los FM

Relación	Semántica Formal	Implicación en la Configuración
Obligatoria	$P \rightarrow C$	Si el padre P es seleccionado, el hijo obligatorio C debe incluirse. No hay variabilidad en este punto
Opcional	$C \rightarrow P$	El hijo C puede ser incluido o excluido, si se selecciona el padre P. Introduce variabilidad

Continúa en la siguiente página

Cuadro 1.1 – Continuación de la página anterior

Relación	Semántica Formal	Implicación en la Configuración
Agrupación AND	$(P \rightarrow \bigwedge_{i=1}^n C_i) \wedge \bigwedge_{i=1}^n (C_i \rightarrow P)$	Si se selecciona la característica padre P, todos los hijos C_i deben incluirse; cada hijo seleccionado implica la presencia del padre. No introduce variabilidad dentro del grupo
Agrupación OR	$(P \rightarrow \bigvee_{i=1}^n C_i) \wedge \bigwedge_{i=1}^n (C_i \rightarrow P)$	Al seleccionar el padre P, debe elegirse al menos un hijo C_i . Cada hijo seleccionado requiere que el padre esté presente. Sí introduce variabilidad
Agrupación alternativa	$(P \rightarrow \bigvee_{i=1}^n C_i) \wedge \bigwedge_{i \neq j} \neg(C_i \wedge C_j) \wedge \bigwedge_{i=1}^n (C_i \rightarrow P)$	Cuando el padre P se selecciona, exactamente un hijo C_i puede activarse y la elección de cualquier hijo obliga a seleccionar al padre. Sí introduce variabilidad

Las relaciones descritas en la tabla 1.1 constituyen la base semántica sobre la cual se construye cualquier Modelo de Características. Sin embargo, para que esta semántica sea útil en la práctica del modelado, es necesario contar con una notación gráfica estandarizada que permita visualizar de forma intuitiva la estructura jerárquica, las restricciones entre características e identificar los puntos de variabilidad. Esta representación visual proporciona una vista de alto nivel que complementa la especificación formal, pues es la encargada de dotar a los FM de su poder comunicativo y analítico. La figura 1.1 ilustra la simbología convencionalmente aceptada en la literatura para representar las relaciones estructurales de los FM, donde cada símbolo codifica visualmente una de las relaciones semánticas previamente explicadas.

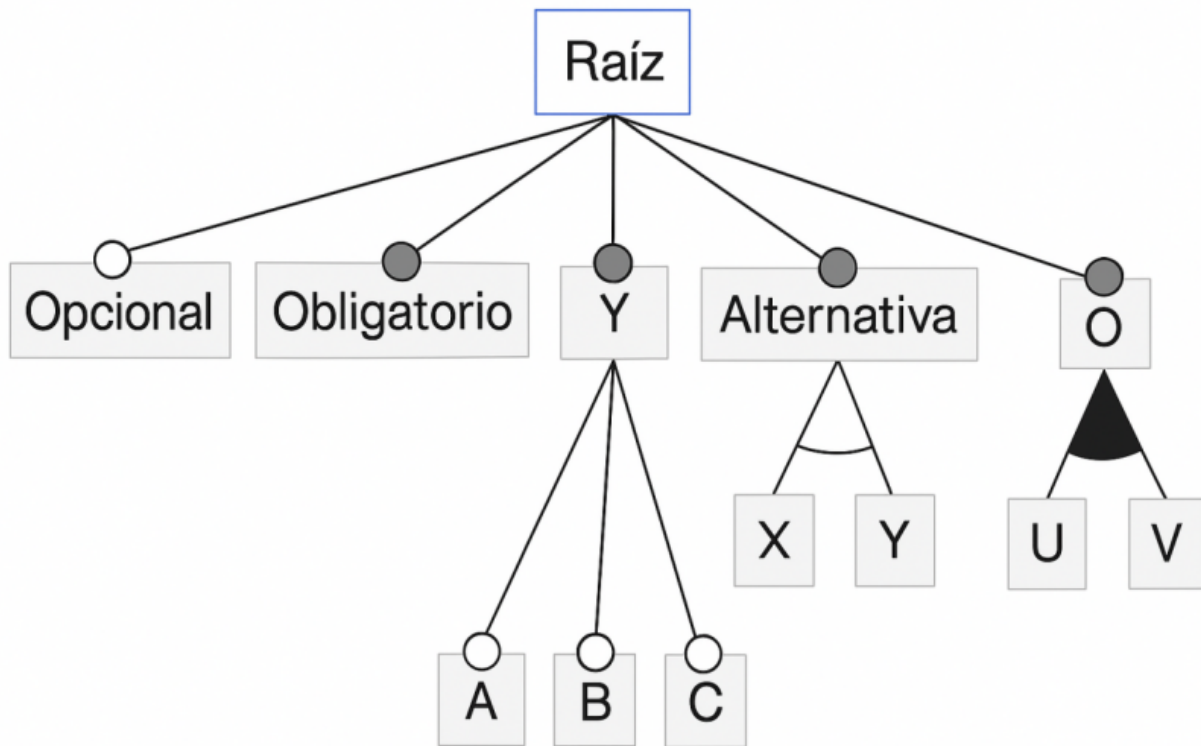


Figura 1.1. Símbolos gráficos para representar las relaciones estructurales.

Restricciones (*Cross-Tree*)

Las relaciones jerárquicas son insuficientes para capturar todas las reglas de negocio complejas. Por ello, los **FM** permiten definir las restricciones transversales (*cross-tree*); que son reglas lógicas que definen dependencias entre características que no están directamente conectadas en la jerarquía padre-hijo del árbol de características. Su función es garantizar la validez de una configuración al capturar las relaciones complejas del mundo real que la estructura jerárquica por sí sola no puede expresar (David Benavides; Trinidad y Antonio Ruiz-Cortés, 2005; C. Sundermann; Dörr; Junker y Kehrer, 2023).

Las dos formas más comunes y fundamentales son:

- **Requiere:** Esta restricción establece una dependencia de presencia. Si una característica A requiere una característica B, entonces siempre que A sea seleccionada en una configuración, B también debe estarlo. Porejemplo, en un **FM** para un automóvil, la característica "Navegación" puede requerir la característica "Pantalla Táctil". Esta relación se modela en lógica proposicional como el condicional $A \rightarrow B$ (David Benavides; Trinidad y Antonio Ruiz-Cortés, 2005; C. Sundermann; Dörr; Junker y Kehrer, 2023).
- **Excluye:** Esta restricción establece un conflicto o incompatibilidad. Si una característica C excluye

una característica D, entonces ambas no pueden formar parte de la misma configuración. Por ejemplo, el motor "Diésel" puede excluir la transmisión eléctrica.^{en} un vehículo híbrido. Su representación lógica es la negación de la conjunción: $\neg(C \wedge D)$ (David Benavides; Trinidad y Antonio Ruiz-Cortés, 2005; C. Sundermann; Dörr; Junker y Kehrler, 2023).

Semántica Formal y Solucionadores SAT (*SAT Solvers*)

La semántica formal de un FM proporciona un significado matemático preciso a todos sus elementos (jerarquía, relaciones y restricciones), lo que permite un análisis automático y libre de ambigüedades.

- Traducción a Lógica Proposicional: La base de la semántica formal es la traducción de todo el FM a una fórmula en lógica proposicional. Cada característica se representa como una variable proposicional (que puede ser Verdadera o Falsa, indicando si está seleccionada o no). Las relaciones padre-hijo y las restricciones cross-tree se convierten en una serie de fórmulas lógicas que, en conjunto, definen el conjunto de todas las configuraciones válidas (C. Sundermann; Dörr; Junker y Kehrler, 2023; T. Thüm; Kästner; Benduhn; Meinicke; Saake y Leich, 2014).
- Satisfacibilidad (*Satisfacibilidad Booleana (SAT)*) y *Conteo de Modelos (SHARPSAT)*: Una vez el FM está representado como una fórmula lógica F, se pueden realizar diversos análisis:
 - Análisis de SAT: Consiste en verificar si existe al menos una asignación de valores (Verdadero/Falso) a las variables que haga que la fórmula F sea verdadera. Esto responde a la pregunta fundamental de si el FM tiene al menos una configuración válida (D. Benavides; S. Segura y A. Ruiz-Cortés, 2010; Mendonca y Wasowski, 2009; C. Sundermann; Dörr; Junker y Kehrler, 2023).
 - Análisis de SHARPSAT: Consiste en el conteo de modelos (*Model Counting*) que satisfacen F. Esto es crucial para determinar el número total de configuraciones válidas que define un FM, una métrica esencial para comprender su variabilidad y complejidad (D. Benavides; S. Segura y A. Ruiz-Cortés, 2010; Mendonca y Wasowski, 2009; C. Sundermann; Dörr; Junker y Kehrler, 2023).
- Solucionadores SAT (*SAT solvers*): Un solucionador SAT (*SAT solver*) es una herramienta algorítmica altamente optimizada diseñada para resolver problemas de satisfacibilidad. Dada una fórmula F, el solucionador (*solver*) determina si es satisfacible (SAT) o insatisfacible (UNSAT). Para el análisis de SHARPSAT, se emplean solucionadores especializados de conteo de modelos (*#SAT solvers*) como sharpSAT y Ganak, que han demostrado un rendimiento excelente al calcular el número de configuraciones en modelos industriales (Soos; Gocht y al., 2020; C. Sundermann; Dörr; Junker y Kehrler, 2023).

Operaciones sobre Modelos de Características

Para que un FM sea útil en un entorno de desarrollo, debe soportar un conjunto de operaciones que permitan manipularlo y consultarlo. Estas operaciones están agrupadas por grupos y son:

- **Operaciones de creación y edición:**

- Creación del modelo: Inicialización de un FM con su característica raíz.
- Inserción/eliminación de características: Añadir o remover nodos en la jerarquía.
- Modificación de relaciones: Cambiar el tipo de relación entre características padre-hijo.
- Gestión de restricciones: Añadir, eliminar o modificar restricciones *cross-tree*.

- **Operaciones de consulta y análisis:**

- Validación del modelo: Verificar la consistencia global del FM.
- Cálculo del número de configuraciones: Determinar la cardinalidad del espacio de productos.
- Extracción de configuraciones: Obtener configuraciones válidas aleatorias o siguiendo criterios específicos.
- Verificación de restricciones: Comprobar si una configuración candidata cumple todas las reglas.

- **Operaciones de transformación:**

- Exportación/importación: Serializar el modelo a formatos estándar (UVL, XML, JSON).
- Refactorización: Reorganizar la estructura del modelo sin alterar su semántica.
- Especialización: Generar un submáximo modelo a partir de decisiones parciales.

Modelos de Características con atributos

Los FM básicos, también conocidos como feature models booleanos, capturan únicamente la presencia o ausencia de características. Sin embargo, muchos dominios requieren modelar información adicional asociada a cada característica, como costos, pesos, tiempos, o en el caso educativo, créditos, horas lectivas o puntuaciones (David Benavides; Trinidad y Antonio Ruiz-Cortés, 2005).

Los FM con atributos extienden el formalismo básico permitiendo asociar a cada característica un conjunto de atributos con valores. Estos atributos pueden ser:

- Numéricos: Enteros o reales (ej., 4 créditos, 2.5 horas).
- Booleanos: Valores verdadero/falso.
- Enumerados: Conjuntos finitos de valores (ej., semestre: 1,2,3,4,5).
- Cadenas: Texto descriptivo.

La verdadera utilidad de los atributos emerge cuando se combinan con restricciones que los involucran. Por ejemplo, en un modelo curricular se pueden expresar reglas como:

- La suma de créditos de las asignaturas optativas no debe exceder 30.
- El costo de los materiales de laboratorio debe ser inferior a \$200.
- La carga horaria semanal debe estar entre 20 y 25 horas.

El análisis de FM con atributos requiere técnicas más sofisticadas, como la programación con restricciones (*constraint programming*) o la verificación de modelos (*model checking*), especialmente cuando se

buscan configuraciones que optimicen funciones objetivo (D. Benavides; S. Segura y A. Ruiz-Cortés, 2010). Lenguajes como UVL incorporan soporte nativo para atributos y restricciones sobre ellos, como se ilustra en el ejemplo del Sándwich 1.1.

Análisis Automatizado de Modelos de Características (AAFM)

La verdadera potencia de los FM reside en la posibilidad de realizar análisis automatizados sobre ellos. El análisis de FM (Análisis Automatizado de Feature Models (AAFM)) es un área de investigación que estudia cómo extraer información útil de un FM mediante algoritmos y herramientas automáticas (ibíd.). Estos análisis permiten responder preguntas fundamentales sobre la línea de productos antes de invertir en su implementación.

D. Benavides; S. Segura y A. Ruiz-Cortés (ibíd.) clasifican los análisis en varias categorías:

- **Análisis de propiedades básicas:** Verifican características elementales del modelo:
 - Consistencia (*consistency*): Determina si existe al menos una configuración válida (modelo no vacío).
 - Modelo muerto (*void model*): Detecta si el FM carece por completo de configuraciones válidas debido a restricciones contradictorias.
 - Características muertas (*dead features*): Identifica características que no pueden ser seleccionadas en ninguna configuración válida.
 - Características falsas opcionales (*false optional features*): Detecta características declaradas como opcionales pero que, debido a restricciones, resultan obligatorias en todas las configuraciones.
- **Análisis de configuración:** Asisten en el proceso de seleccionar características:
 - Validación de configuraciones: Verifica si una selección parcial o completa de características constituye una configuración válida.
 - Propagación de decisiones: Calcula automáticamente las consecuencias de seleccionar o excluir una característica.
 - Explicación de inconsistencias: Identifica las restricciones responsables de que una configuración sea inválida.
- **Análisis de optimización:** Buscan configuraciones que maximicen o minimicen ciertos criterios:
 - Selección óptima de características: Encuentra configuraciones que optimicen funciones objetivo (costo, rendimiento, calidad).
 - Análisis multi-objetivo: Considera múltiples criterios simultáneamente para encontrar el conjunto de configuraciones Pareto-óptimas.

1.1.3. Lenguaje Universal de Variabilidad (UVL)

La diversidad de notaciones y lenguajes para el modelado de la variabilidad ha sido históricamente un obstáculo para la colaboración científica y la transferencia tecnológica en el área de [SPL](#). Ante esta situación, y como resultado de una iniciativa comunitaria denominada MODEVAR, surgió el Lenguaje Universal de Variabilidad (*Universal Variability Language* - [UVL](#)), un esfuerzo colaborativo que ha involucrado a investigadoras e investigadores de universidades de España, Alemania, Austria y otros países durante más de seis años (David Benavides; Chico Sundermann; Feichtinger; Galindo; Rabiser y Thomas Thüm, [2025](#); Chico Sundermann; Feichtinger; Engelhardt; Rabiser y Thomas Thüm, [2021](#)).

El [UVL](#) se define como un lenguaje abierto e independiente de plataforma, diseñado específicamente para especificar modelos de variabilidad de forma textual. Su principal objetivo es proporcionar una representación común que pueda ser utilizada tanto en el ámbito académico como en la industria, facilitando así el intercambio de modelos, la comparación de resultados y el desarrollo de herramientas interoperables (UVL Community, [\[s.f.\]](#)). La creación de [UVL](#) responde a la necesidad de superar la fragmentación provocada por la existencia de múltiples lenguajes propietarios o académicos con capacidades y expresividad heterogéneas.

Aunque [UVL](#) nace en el contexto del software, su naturaleza genérica lo hace aplicable a cualquier dominio que requiera modelar variabilidad. El laboratorio *Diverso Lab* de la Universidad de Sevilla, liderado por el catedrático David Benavides, ha aplicado [UVL](#) en dominios tan diversos como las aplicaciones móviles, la automatización del marketing, la personalización de sensores para sistemas de riego, y recientemente, en la exploración de su aplicación al ámbito educativo para el modelado de currículos flexibles (Oficina de Transferencia de Resultados de Investigación - Universidad de Sevilla, [2025](#)). Esta versatilidad convierte a [UVL](#) en una opción particularmente adecuada para la presente investigación, donde se utilizará como notación base para representar la variabilidad curricular.

Diseño del lenguaje y niveles de expresividad

[UVL](#) especifica modelos de variabilidad con una estructura similar a un árbol para representar la estructura jerárquica de los modelos de variabilidad. El núcleo del lenguaje viene con varios conceptos para especificar restricciones, y son exactamente los mismos que se explican en la tabla [1.1](#) junto a las limitaciones de árboles cruzados (restricciones *cross-tree*).

El diseño de [UVL](#) se estructura en torno, además de su núcleo, en un sistema de niveles lingüísticos (*language levels*) que permiten incrementar su expresividad de forma controlada. Su sintaxis básica representa la estructura jerárquica de un [FM](#) mediante una notación de árbol indentado, similar a la utilizada en lenguajes de programación como Python (Chico Sundermann; Vill; Thomas Thüm; Feichtinger; Agarwal; Rabiser; Galindo y David Benavides, [2023](#)).

El lenguaje contempla los siguientes niveles principales (UVL Community, [\[s.f.\]](#)):

- **Nivel Booleano (*Boolean Level*):** Constituye la base del lenguaje e incluye:

- *Núcleo Booleano (Boolean Core)*: Soporta las relaciones fundamentales (obligatoria, opcional, OR, alternativa) y restricciones transversales proposicionales.
- *Cardinalidad de Grupo (Group Cardinality)*: Extiende las relaciones padre-hijo permitiendo especificar que se seleccionen entre n y m características hijas.
- **Nivel Aritmético (Arithmetic Level)**: Introduce la posibilidad de trabajar con atributos numéricos asociados a las características:
 - *Núcleo Aritmético (Arithmetic Core)*: Permite definir restricciones sobre atributos mediante operaciones aritméticas estándar (+, -, *, /, =, !=, >, <).
 - *Agregados de Atributos (Attribute aggregates)*: Simplifica la especificación de restricciones globales mediante funciones como `sum()` y `avg()`.
 - *Cardinalidad de Característica (Feature Cardinality)*: Permite que una misma característica pueda ser seleccionada múltiples veces dentro de un rango $[n..m]$.
- **Nivel Tipado (Type Level)**: Amplía el lenguaje con tipos de datos más complejos:
 - *Núcleo Tipado (Type Core)*: Introduce características de tipo cadena (*String Features*).
 - *Restricciones Numéricas (Numeric-Constraints)*: Añade operaciones como `floor()` y `ceil()` dentro de las restricciones.
 - *Restricciones de Cadena (String-Constraints)*: Permite operaciones como `len()` y comparación entre cadenas.

Además, [UVL](#) incorpora un mecanismo de importación que permite referenciar submodelos, facilitando así la composición de modelos de variabilidad de gran tamaño a partir de módulos más pequeños y gestionables (UVL Community, [\[s.f.\]](#)). El FM representado (1.1) ejemplifica la sintaxis [UVL](#) y la correspondencia con su representación gráfica (1.2):

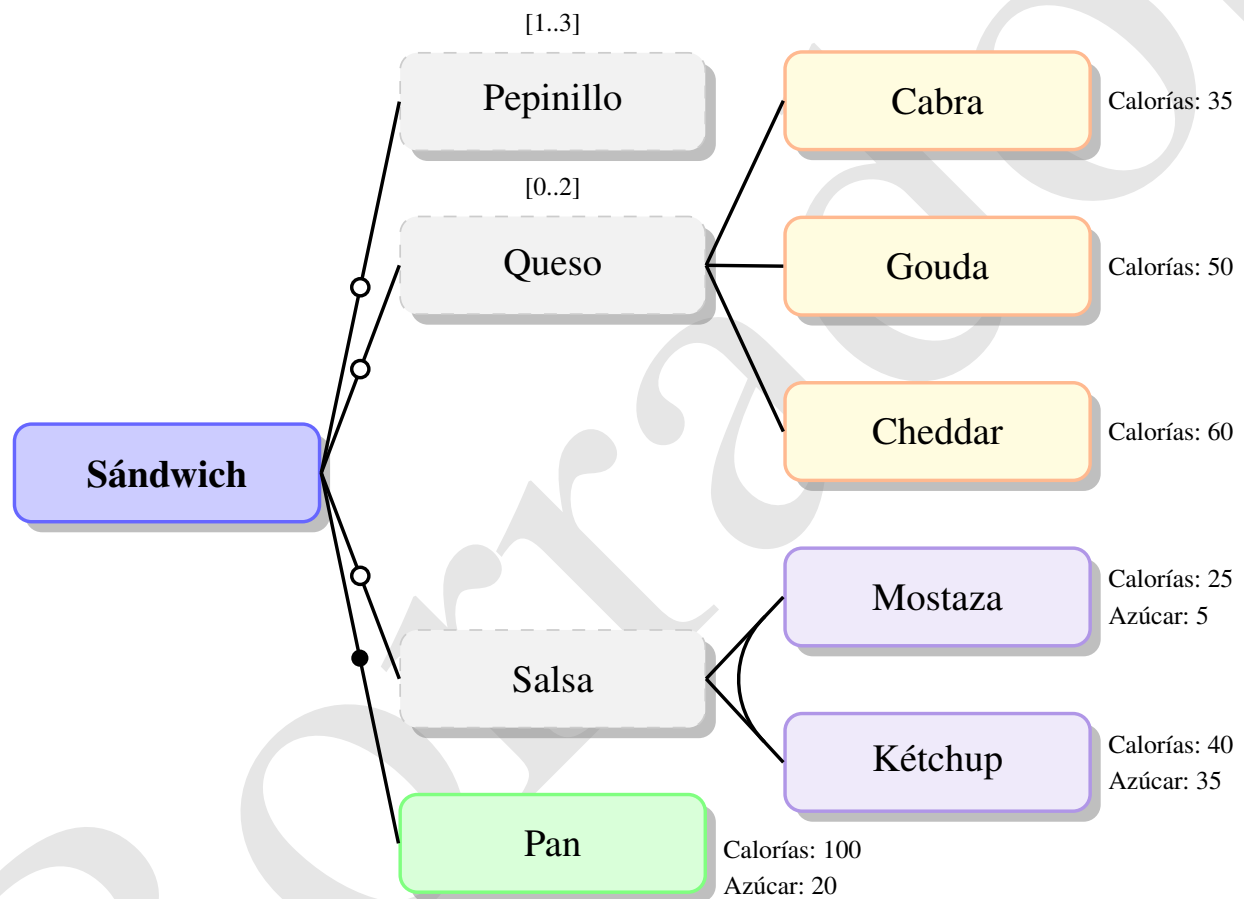
Código fuente 1.1. Ejemplo de modelo de variabilidad en [UVL](#)

```
include
  Boolean.group-cardinality
  Arithmetic.aggregate-function
  Arithmetic.feature-cardinality
  Type
features
  Sandwich
    mandatory
      Pan {Calorias 100, Azucar 20}
    optional
      Salsa
        or
          Ketchup {Calorias 40, Azucar 35}
          Mostaza {Calorias 25, Azucar 5}
      Queso
```

```

[0..2] // Group cardinality
  Cheddar {Calorias 60}
  Gouda {Calorias 50}
  Cabra {Calorias 35}
  Pepinillo cardinality [1..3] // Feature cardinality
constraints
  Ketchup => Queso
  Pan.Azucar + Ketchup.Azucar + Mostaza.Azucar < 60 // Attribute constraints
  sum(Calorias) < 160 // Attribute aggregate

```



ID	Restricciones Cruzadas (UVL / Lógica Proposicional)
R1	Ketchup $\Rightarrow$ Queso
R2	Pan.Azúcar + Ketchup.Azúcar + Mostaza.Azúcar $\leq$ 60
R3	sum(Calorías) < 60

Figura 1.2. Representación gráfica del FM del modelo Sandwich especificado en UVL. Fuente: elaboración propia.

1.1.4. Configuraciones y configurador curricular

En el contexto de las Líneas de Productos de Software (SPL), la configuración se refiere al proceso sistemático de seleccionar un conjunto específico de características de un FM para derivar un producto individual que satisfaga requisitos particulares (Apel; Batory; Kästner y Saake, 2013). Este proceso implica tomar decisiones de configuración que respeten todas las restricciones definidas en el modelo, asegurando que la selección final de características sea válida y consistente (D. Benavides; S. Segura y A. Ruiz-Cortés, 2010).

La configuración en SPL se caracteriza por ser un proceso guiado por restricciones donde las dependencias entre características (mandatorias, opcionales, alternativas) y las restricciones cross-tree (requiere, excluye) determinan el espacio de configuraciones posibles (Chen; Babar y Ali, 2020). La complejidad de este proceso aumenta exponencialmente con el tamaño del modelo, haciendo necesario el uso de herramientas automatizadas para garantizar la corrección de las configuraciones generadas (D. Benavides; S. Segura y A. Ruiz-Cortés, 2010).

Un configurador curricular es una herramienta especializada que aplica los principios de configuración de SPL al dominio de la planificación educativa, permitiendo la generación semi-automatizada de planes de estudio personalizados a partir de un modelo base que captura la variabilidad curricular (Sánchez; Gutiérrez y Cabrera, 2020). Este sistema opera como un motor de decisiones académicas que guía al usuario (gestor académico, coordinador de programa) a través del espacio de configuraciones válidas, garantizando que el plan de estudio resultante cumpla con todas las restricciones pedagógicas y administrativas (Chen; Babar y Ali, 2020).

El valor fundamental de un configurador curricular reside en su capacidad para reducir la complejidad del diseño curricular mediante la aplicación de restricciones formales que previenen configuraciones inválidas (Rodríguez; Vizcaíno y Piattini, 2019). Por ejemplo, el sistema puede asegurar automáticamente que:

- Se cumplan todos los prerrequisitos entre asignaturas y temáticas de las mismas.
- No existan asignaturas y/o temáticas que no esten relacionadas con otras o que sí lo estén, pero debido a las restricciones y reglas, es imposible seleccionar o incluir en cualquier configuración válida del producto.
- Los límites de créditos por semestre se respeten.
- La coherencia entre las competencias a desarrollar y las asignaturas seleccionadas se mantengan.
- Se eviten conflictos de horario y sobrecarga académica.
- No existan asignaturas o temáticas incompatibles en el mismo plan de estudio.
- Se garantice la inclusión de asignaturas obligatorias y la correcta selección de asignaturas optativas según las reglas definidas en el modelo curricular.
- Se cumplan las restricciones de diversidad temática y metodológica para asegurar un plan de estudio equilibrado y completo.

1.1.5. Gestión y flexibilidad curricular. Línea de Productos Educativos

La gestión curricular, como se expuso al inicio de este capítulo, constituye el conjunto de procesos mediante los cuales las instituciones educativas diseñan, implementan, evalúan y actualizan sus planes de estudio. Tradicionalmente, esta gestión se ha apoyado en modelos rígidos y documentos estáticos que dificultan la adaptación a contextos cambiantes (Gómez, 2023). Sin embargo, las demandas del siglo XXI —caracterizadas por la aceleración tecnológica, la diversidad de perfiles estudiantiles y la necesidad de formación continua— exigen un enfoque radicalmente distinto: la flexibilización curricular.

La flexibilidad curricular puede entenderse como la capacidad de un plan de estudios para adaptarse a las necesidades, intereses y contextos de los estudiantes o bien por necesidades circunstanciales, así como a las demandas del entorno social y productivo, sin perder coherencia ni calidad académica (Díaz, 2002). Esta flexibilidad opera en múltiples dimensiones:

- Flexibilidad en el tiempo: Posibilidad de cursar asignaturas en diferentes momentos, ritmos y secuencias.
- Flexibilidad en el espacio: Modalidades presencial, híbrida o virtual.
- Flexibilidad en los contenidos: Selección de asignaturas optativas, énfasis, menciones o itinerarios formativos.
- Flexibilidad metodológica: Diversidad de enfoques pedagógicos (aprendizaje basado en proyectos, aula invertida, gamificación).
- Flexibilidad en la evaluación: Adaptación de los mecanismos de acreditación a las características del estudiante y del contexto.

Lograr esta flexibilidad de manera sistemática y sostenible requiere superar las prácticas artesanales basadas en documentos y hojas de cálculo. Es aquí donde los paradigmas de la ingeniería de software ofrecen un camino prometedor.

Hacia las Líneas de Productos Educativos

La aplicación del paradigma de SPL al dominio educativo da lugar al concepto de Líneas de Productos Educativos. Esta puede definirse como una familia de artefactos educativos (planes de estudio, cursos, materiales didácticos, actividades de aprendizaje) que comparten un núcleo común y que pueden ser configurados sistemáticamente para generar productos adaptados a contextos, perfiles o necesidades específicas (Chimalakonda y Nori, 2012; Okada; Hisazumi; Nakanishi y Fukuda, 2009).

La idea fundamental consiste en trasladar los principios de la SPLE —gestión de variabilidad, reutilización planificada, activos centrales, puntos de variación— al diseño y gestión curricular. Así, un tronco curricular común (asignaturas obligatorias, competencias básicas, estructura general) constituye los activos centrales de la línea, mientras que los itinerarios formativos, las asignaturas optativas, las menciones o las modalidades de impartición se modelan como puntos de variabilidad (Silva Marcolino y Francine Barbosa, 2017).

Investigaciones pioneras en este campo han demostrado la viabilidad y los beneficios de este enfoque:

- Okada; Hisazumi; Nakanishi y Fukuda (2009) propusieron una metodología para mejorar la reutilización de materiales educativos mediante ingeniería de líneas de productos. Su trabajo demostró que la separación de partes comunes y variables en los materiales docentes permite generar variantes adaptadas a diferentes escenarios de uso, resolviendo los problemas de consistencia que surgen cuando los educadores gestionan múltiples variantes de forma artesanal.
- Chimalakonda y Nori (2012) exploraron la aplicación de SPL en tecnologías educativas para abordar la diversidad lingüística y contextual de la India. Su investigación evidenció que el desarrollo de soluciones educativas para 22 idiomas y múltiples contextos socioculturales puede beneficiarse enormemente de un enfoque sistemático de reutilización, reduciendo el esfuerzo de desarrollo de años-persona a procesos de configuración guiados.
- Silva Marcolino y Francine Barbosa (2017) diseñaron una arquitectura de línea de productos para construir aplicaciones de aprendizaje móvil (m-learning) orientadas a la enseñanza de programación. Su trabajo identificó las características comunes y variables en este dominio y demostró cómo el enfoque SPL puede facilitar la creación de herramientas educativas adaptadas a diferentes estilos de aprendizaje y contextos institucionales.
- Solis Pino; Ruiz y Hurtado Alegria (2022) aplicaron SPL como herramienta educativa para el aprendizaje de programación de robots industriales con Arduino. Su estudio con estudiantes de primer año universitario mostró que el enfoque permitió reutilizar entre el 38 % y el 43 % del código total necesario para programar los robots, mejorando significativamente el tiempo de desarrollo y facilitando el aprendizaje de conceptos de abstracción y modularidad.
- Galán y otros (2014) refactorizaron una aplicación de e-learning (E-Learning Web Miner) como una línea de productos, demostrando que el paradigma SPL puede aliviar los costosos y propensos a errores procesos de adaptación manual que caracterizan a muchas plataformas educativas cuando deben ajustarse a diferentes contextos de enseñanza-aprendizaje.

1.2. Fortalezas de los Modelos de Característica aplicado en la gestión curricular

La verdadera potencia del FM radica en que, una vez formalizado, se convierte en un modelo lógico-matemático que define explícitamente el espacio de diseño o el conjunto de configuraciones válidas (2^n) que se pueden generar (donde n es el número de características). Esta formalidad permite realizar análisis automatizados esenciales mediante herramientas conocidas como solucionadores de satisfacibilidad (SAT Solvers) o restricciones, permitiendo la validación de consistencia, que se encarga de determinar si un FM es consistente, es decir, si existe al menos una configuración válida; la Identificación de características muertas, lo cual permite encontrar las características que nunca pueden ser seleccionadas debido a la severidad de las

restricciones y posibilitan una configuración guiada, garantizándose que el producto final (ej. el plan de estudios) sea siempre válido.

Aunque existen otros modelos de variabilidad (como las tablas de decisión o los diagramas UML de variabilidad), el FM es el formalismo preferido por su simplicidad visual y su potencia analítica. A diferencia de modelos como las tablas, que pueden volverse inmanejables al aumentar el número de características, el FM provee una visión de alto nivel y, simultáneamente, una base lógica binaria robusta para el análisis automatizado de la consistencia.

En los próximos epígrafes se profundiza en esta línea argumental desde dos perspectivas complementarias. En primer lugar, se establece una analogía formal curricular que traduce los principales conceptos de la SPLE al dominio educativo, demostrando mediante un ejemplo (1.3 y 1.4) concreto —como el modelado de la asignatura Estructura de Datos I— cómo los FM pueden capturar la variabilidad intrínseca de los planes de estudio. Posteriormente, se sintetizan los beneficios de las Líneas de Productos Educativos, destacando las ventajas que este enfoque aporta a la gestión curricular en términos de reutilización, consistencia, trazabilidad y exploración del espacio de configuraciones.

1.2.1. Analogía formal curricular

La aplicación de los FM a la gestión y diseño curricular constituye el principal aporte metodológico de esta investigación. Esta extensión se fundamenta en la premisa de que la variabilidad, concepto central de la SPLE es intrínseca y necesaria en los planes de estudio modernos de la enseñanza superior.

Para fundamentar la propuesta, es imprescindible establecer una analogía 1.2 formal y coherente que permita sustituir los constructos de SPL por sus equivalentes en el dominio educativo, elevando el plan de estudios de un simple documento descriptivo a un activo configurable y analizable:

Cuadro 1.2. Analogía de conceptos de SPL en el Dominio Educativo. Fuente: Elaboración propia.

Concepto de SPL	Concepto análogo en planificación curricular	Justificación
Línea de Productos	Carrera o área de estudio	El tronco común de conocimientos que define una familia de perfiles profesionales
Producto Específico	Plan de estudio configuracional	La ruta académica única y válida para un estudiante o perfil de egreso definido
Característica	Unidad curricular(asignatura, módulo, tema de estudio)	La unidad funcional mínima que puede ser seleccionada u omitida
Característica Raíz	El currículo base (ej. la carrera Ingeniería en Ciencias Informáticas o en caso que se esté modelando una asignatura, el nombre de la misma)	La entidad que engloba todos los módulos y asignaturas posibles de la carrera

Continúa en la siguiente página

Cuadro 1.2 – Continuación de la página anterior

Concepto de SPL	Concepto análogo en planificación curricular	Justificación
Variabilidad	Opciones académicas	El conjunto de decisiones disponibles (optatividad, electividad, elección de menciones) que permiten personalizar la ruta del estudiante

Yendo un paso más adelante para evidenciar la aplicabilidad del FM al dominio curricular, se presenta un ejemplo (figuras 1.3 y 1.4) basado en la asignatura Estructura de Datos I. Se expone el código 1.2 UVL y su esquema jerárquico conceptual con el fin ilustrativo y suficiente para comprender las decisiones de variabilidad del FM:

Código fuente 1.2. Ejemplo de modelo de variabilidad en UVL

```

namespace EstructuraDatos
include
    Boolean.group-cardinality
    Arithmetic.aggregate-function
    Arithmetic.feature-cardinality
    Type
features
    Estructura de Datos I
        mandatory
            Fundamentos y Análisis {Horas 6}

            Estructuras Lineales
                cardinality [1..*]
                mandatory
                    Arreglos
                    Listas Enlazadas
                optional
                    Listas Circulares

            Pilas {Horas 6}
                mandatory
                    Implementacion
                        alternative
                            Con Arreglos {Horas 4, Dif 1}
                            Con Listas {Horas 6, Dif 2}

            Colas {Horas 8}
                mandatory
                    FIFO {Dif 2}
                    Prioridad {Dif 3}
                optional

```

```

Deque {Dif 2}

Árboles Binarios {Horas 12, Dif 4}

Algoritmos Ordenamiento {Horas 10}

Lenguaje
  alternative
    C/C++ {Carga 9}
    Java {Carga 6}
    Python {Carga 4}

Técnicas de Programación
  mandatory
    Recursión Avanzada {Carga 8}
    Generica {Carga 10}
    Manejo de Memoria {Carga 8}

Herramientas
  or
    IDE {Carga 5}
    Debugger {Carga 2}
    Git {Carga 4}

Análisis de Complejidad
  mandatory
    Notación Big-0
    Análisis Temporal
    Análisis Espacial

constraints
  // R1: Prerrequisito pedagógico
  Árboles Binarios => Recursión Avanzada

  // R2: Dependencia técnica de bajo nivel
  C/C++ <=> Manejo de Memoria

  // R3: Restricción por abstracción del lenguaje
  Python => !Manejo de Memoria

  // R4: Restricción de recursos (Suma de atributos)
  sum(Horas) <= 64

  // R5: Restricción de viabilidad (Cálculo entre atributos)
  (avg(Dif) * Lenguaje.Carga) < 50

```

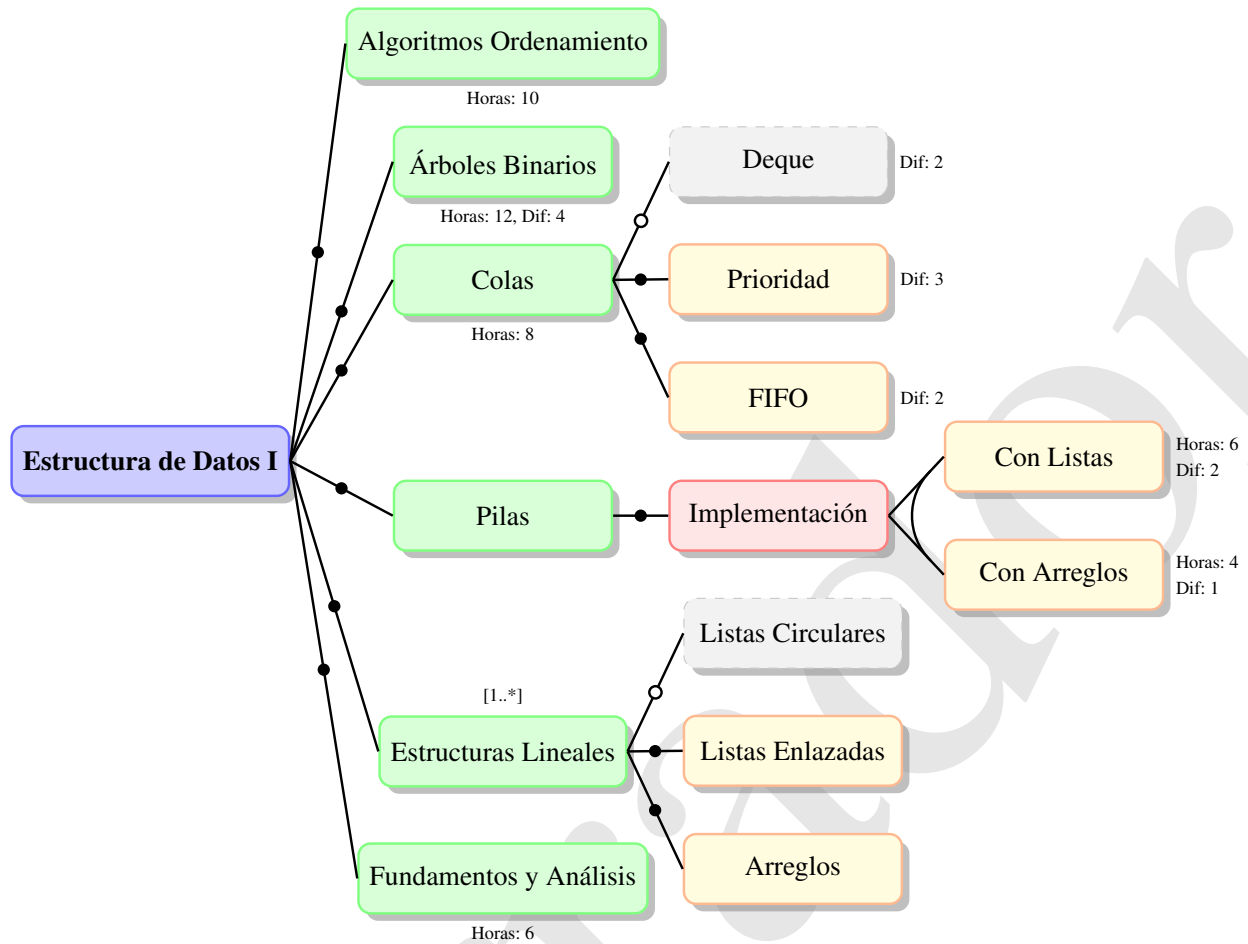
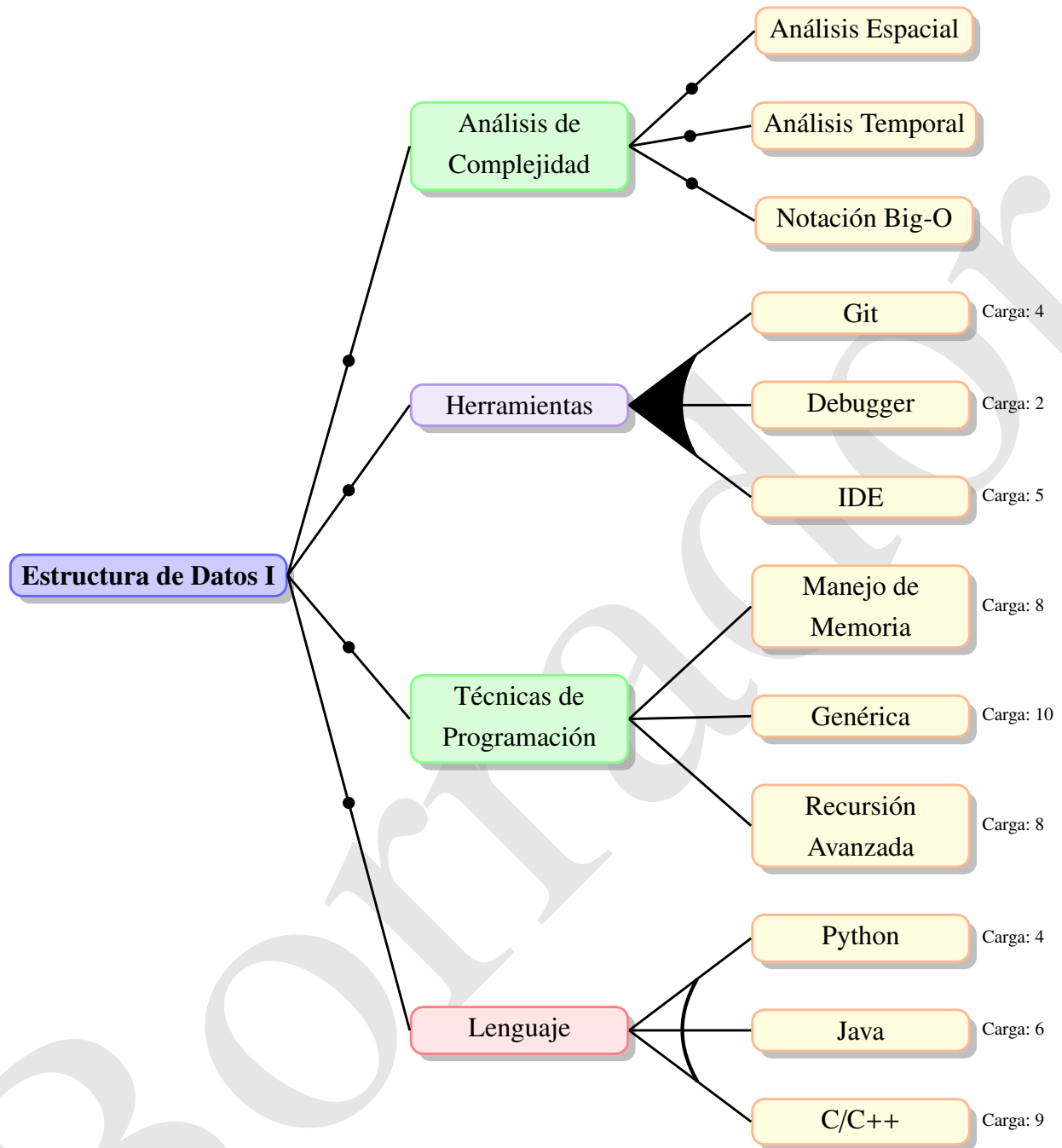


Figura 1.3. Representación simplificada #1 del FM de la asignatura Estructura de Datos I. Fuente: elaboración propia.



ID	Restricciones Cruzadas (UVL / Lógica Proposicional)
R1	Árboles Binarios $\Rightarrow$ Recursión Avanzada
R2	C/C++ $\Leftrightarrow$ Manejo de Memoria
R3	Python $\Rightarrow \neg$ Manejo de Memoria
R4	$\text{sum}(\text{Horas}) \leq 64$ (Límite semestral)
R5	$\text{avg}(\text{Dificultad}) * \text{Lenguaje.Carga} < 50$ (Umbral de viabilidad)

Figura 1.4. Representación simplificada #2 del FM de la asignatura Estructura de Datos I. Fuente: elaboración propia.

Este caso permite demostrar cómo una asignatura, lejos de ser un listado estático de contenidos, posee una estructura jerárquica, elementos obligatorios y opcionales, variabilidad controlada, dependencias de prerequisites y restricciones internas; todas ellas propiedades capturables dentro de un FM formal. El modelo mostrado (1.3 y 1.4) constituye una representación declarativa de la asignatura, estructurada como un árbol donde la característica raíz corresponde al curso completo y las características hijas representan módulos, temas o decisiones didácticas. Esta representación no solo captura los contenidos, sino que los organiza con semántica formal, permitiendo análisis automatizable. Los ejemplos ilustra distintos tipos de variabilidad presentes en currículos reales: contenidos obligatorios (módulos troncales), opcionales (tópicos avanzados), alternativas mutuamente excluyentes (implementación de pilas), configuraciones tipo OR (técnicas complementarias) y variantes dependientes (manejo de memoria solo si la implementación se realiza en C/C++).

La estructura del FM muestra claramente cómo una asignatura puede contener subniveles jerárquicos complejos, donde cada nivel constituye un subconjunto configurable. Esto permite modelar asignaturas que se imparten según distintas líneas, enfoques o lenguajes de implementación (C/C++, Java, Python), conservando la validez curricular y evitando inconsistencias.

La variabilidad no solo define qué temas se abordan, sino cómo se implementan, qué carga cognitiva poseen y qué prerequisites deben cumplirse. La presencia de relaciones cruzadas de árboles, restricciones lógicas e implicaciones capturan dependencias reales, tales como: para comprender árboles es necesario estudiar recursión (relación R1), o bien optar por C/C++ implica la posibilidad de manejo manual de memoria (relación R2). Esta lógica curricular, que en los planes tradicionales queda implícita o depende de interpretación docente, aquí se vuelve explícita, formal y verificable. Además, la inclusión de atributos numéricos (Horas, Dificultad, Carga) permite que el FM funcione como un sistema de soporte a la decisión. Restricciones como R4 y R5 demuestran cómo el formalismo puede garantizar que una configuración curricular no solo sea lógicamente válida, sino también factible en términos de carga cognitiva para el estudiante y disponibilidad horaria del programa.

De este modo, la asignatura deja de ser un documento textual estático para convertirse en un componente configurable. El FM presenta la asignatura como un micro-espacio de variabilidad curricular, análogo a un “producto configurable” dentro de una línea de productos académicos. Diferentes configuraciones del FM representan distintas adaptaciones válidas de la asignatura según perfiles de estudiante, competencias, niveles de profundidad o enfoques pedagógicos. Este modelo puede procesarse mediante un motor lógico para verificar su consistencia, detectar características inaccesibles y generar configuraciones válidas automáticamente.

Por tanto, los FM no solo son adecuados para representar planes de estudios completos, sino también para modelar unidades académicas individuales con variabilidad interna. Esto abre la puerta a una flexibilidad curricular controlada, verificable y sustentada en formalismos computacionales, alineándose con los principios de SPLE aplicados a la planificación educativa.

1.2.2. Beneficios de las Líneas de Productos Educativos

En una Línea de Productos Educativos, la variabilidad curricular se convierte en el objeto central de modelado y gestión. Retomando los conceptos de [FM](#) y [UVL](#) desarrollados en secciones anteriores, es posible representar explícitamente:

- Características obligatorias: Asignaturas troncales, competencias básicas, prácticas profesionales obligatorias.
- Características opcionales: Asignaturas optativas, seminarios complementarios, actividades extracurriculares.
- Relaciones de alternancia: Menciones o itinerarios excluyentes entre sí (ej., mención en Ingeniería de Software vs. mención en Inteligencia Artificial).
- Agrupaciones OR: Conjuntos de asignaturas entre las cuales debe elegirse un mínimo (ej., elegir al menos 3 de 5 asignaturas de un bloque).
- Restricciones (*cross-tree*): Prerrequisitos entre asignaturas, incompatibilidades, correlatividades, límites de créditos.
- Atributos: Créditos, horas lectivas, semestre recomendado, área de conocimiento, modalidad de impartición.

La adopción del enfoque [SPL](#) en la gestión curricular reporta beneficios análogos a los observados en el desarrollo de software (Apel; Batory; Kästner y Saake, [2013](#)). De los cuales se tiene:

- Reutilización sistemática: Los diseños curriculares, una vez validados, pueden ser reutilizados para generar nuevos programas académicos sin partir desde cero.
- Consistencia y trazabilidad: La representación formal de la variabilidad garantiza que todos los planes de estudio derivados respeten las mismas reglas y restricciones, facilitando la trazabilidad de los cambios.
- Reducción de errores: Al automatizar la validación de configuraciones, se eliminan los errores humanos derivados de la gestión manual (inconsistencias, omisiones, conflictos).
- Agilidad en la actualización: Las modificaciones al modelo central se propagan automáticamente a todos los productos derivados, manteniendo la coherencia institucional.
- Exploración del espacio de configuraciones: Herramientas de análisis automatizado pueden responder preguntas como “¿cuántos planes de estudio válidos pueden generarse?” o “¿qué asignaturas quedan sin posibilidad de ser cursadas?”.

1.3. Análisis del mercado

El presente epígrafe desarrolla un análisis sistemático del mercado relacionado con los sistemas de gestión curricular y herramientas de modelado de variabilidad. Se examinan tanto las soluciones tradicionales

empleadas por instituciones académicas como aquellas provenientes del ámbito de la ingeniería de líneas de productos, identificando sus fortalezas, limitaciones y brechas funcionales. A partir de esta revisión, se fundamenta la oportunidad tecnológica que sustenta la propuesta investigativa, destacando el vacío existente entre las plataformas de administración académica y los mecanismos de planificación curricular basados en variabilidad, lo que justifica la necesidad y pertinencia de la solución desarrollada.

1.3.1. Panorama General del Mercado

El mercado de sistemas de gestión curricular presenta una segmentación clara entre soluciones genéricas de administración académica y herramientas especializadas en modelado de variabilidad. Mientras plataformas como Moodle, Blackboard y Canvas dominan el segmento de los [Sistema de Gestión del Aprendizaje \(LMS\)](#) (Anthology Inc., 2025; Instructure, 2025; Moodle Project, 2025), existen vacíos significativos en soluciones específicas para la planificación curricular basada en Líneas de Productos Educativos (Sánchez; Gutiérrez y Cabrera, 2020).

1.3.2. Análisis de Soluciones Existentes

El análisis de soluciones existentes se organiza en dos grupos claramente diferenciados, cada uno representativo de perspectivas y alcances funcionales opuestos dentro del dominio investigado. Por un lado, se examinan los sistemas institucionales de gestión curricular tradicional, ampliamente implementados en universidades, cuyo propósito principal es administrar información académica pero que presentan limitaciones para modelar y validar variabilidad curricular. Por otro lado, se estudian herramientas avanzadas de ingeniería de líneas de productos de software ([SPL](#)), diseñadas para gestionar características y restricciones complejas, pero que requieren conocimientos técnicos elevados y carecen de adaptación al contexto educativo. Esta comparación permite identificar brechas tecnológicas y conceptuales que fundamentan la necesidad de una solución especializada que integre fortalezas de ambos mundos mientras atiende las demandas propias de la planificación curricular.

Sistemas de Gestión Curricular Tradicionales

- SIGA y Banner: Los sistemas como Banner son fundamentalmente sistemas de información transaccionales diseñados para la gestión operativa de una institución. Su arquitectura está optimizada para manejar grandes volúmenes de datos de estudiantes, matrículas, registros de cursos y pagos. La evidencia de su funcionamiento, como se observa en la documentación de "Banner Student", muestra una estructura basada en bloques, ventanas y secciones para la gestión de admisiones, currículos y campos de estudio (Ellucian Company, 2015b) (Ellucian Company, 2015). Esta lógica está orientada a capturar y recuperar información de forma consistente, no a modelar relaciones complejas y variables entre componentes curriculares. Carecen de un motor de reglas nativo que permite definir y verificar restricciones complejas como las que se modelan con “requiere” o “excluye” en un [FM](#). Su limita-

ción principal es que operan sobre una estructura de datos fija y predefinida, lo que los hace rígidos para experimentar con diferentes configuraciones curriculares o para gestionar una línea de productos educativos donde las relaciones entre asignaturas y competencias son dinámicas y variables.

- Moodle: como [LMS](#), es una plataforma excelente para la ejecución y gestión de un plan de estudios ya definido. Su fortaleza reside en organizar cursos, desglosarlos en unidades y lecciones, gestionar la entrega de contenidos, las actividades de evaluación y la interacción entre estudiantes y profesores (Moodle Project, [2025](#)). Sin embargo, su funcionalidad no se extiende al diseño instruccional ni a la planificación curricular basada en características. Moodle asume que el diseño del curso (los objetivos, la secuencia de contenidos, las actividades de evaluación) ya ha sido realizado externamente. No proporciona herramientas para que los gestores académicos modelen visualmente un plan de estudios, definan características opcionales u obligatorias, o establezcan restricciones entre ellas. En el contexto de la investigación, Moodle sería un consumidor potencial de la configuración curricular final generada por la propuesta de solución, pudiéndose integrar como plataforma de entrega y gestión de los cursos derivados, pero no como herramienta para diseñar o validar la estructura curricular en sí.

Herramientas de Modelado de Variabilidad

- Pure::variants: es una solución comercial robusta para [SPLE](#). Proporciona un entorno completo para definir características, crear modelos, gestionar restricciones y, crucialmente, derivar productos concretos a partir de configuraciones válidas. Su potencia es innegable en el dominio técnico para el que fue diseñado (pure-systems GmbH, [2024](#)). El problema central para el contexto educativo es su alta barrera de entrada para usuarios no técnicos. Los gestores académicos y diseñadores curriculares, que son expertos en pedagogía pero no necesariamente en [SPLE](#), se verían enfrentados a una terminología y flujos de trabajo propios de la ingeniería de software. Adaptar pure::variants requeriría una personalización significativa de la interfaz y una capacitación intensiva, lo que dificulta su adopción ágil en un entorno académico, validando la necesidad de una solución más especializada y accesible.
- FeatureIDE: es un entorno de código abierto construido como un complemento (*plugin*) para Eclipse, lo que lo hace muy popular en entornos de investigación y enseñanza de la ingeniería de software (T. Thüm; Kästner; Benduhn; Meinicke; Saake y Leich, [2014](#)). Al igual que pure::variants, es poderoso para el análisis y la configuración de modelos de características. Sin embargo, su dependencia de Eclipse y su orientación hacia desarrolladores lo convierten en una herramienta que requiere especialización técnica. Para un coordinador de carrera o un vicerrector académico, interactuar con FeatureIDE supondría una curva de aprendizaje muy pronunciada. Su arquitectura no está diseñada para integrarse de manera natural con los sistemas de información universitarios existentes (como los propios SIGA o Banner), lo que limita su utilidad práctica para la gestión curricular del día a día. Su uso en educación se limita predominantemente a la enseñanza de conceptos de [SPL](#), no a la gestión operativa de currículos.
- Flamapy / FlamapyIDE: es un ecosistema de código abierto para el análisis automatizado de [FM](#), desa-

rollado activamente por el grupo de investigación Diverso Lab de la Universidad de Sevilla, liderado por el catedrático David Benavides (Galindo; Horcas; Felferning; Fernandez-Amoros y David Benavides, 2023). A diferencia de pure::variants y FeatureIDE, Flamapy se concibe desde su origen como un “*framework Python*” modular y extensible, diseñado para facilitar la integración de capacidades de análisis de variabilidad en otras aplicaciones, lo que lo emparenta directamente con el objetivo de la presente investigación. El framework soporta múltiples formatos de entrada, incluyendo UVL, FeatureIDE (.fide) y otros (Team, 2025a), y proporciona transformaciones a representaciones intermedias como diagramas de decisión binaria (BDD) para análisis eficientes (Team, 2025c).

Flamapy ofrece tres interfaces principales que lo convierten en una propuesta particularmente relevante (Team, 2024):

- FlamapyIDE: un entorno de edición y análisis basado en web, accesible en <https://ide.flamapy.org/editor>, que permite cargar modelos UVL o FeatureIDE, visualizar su estructura, y ejecutar operaciones de análisis (validación de consistencia, detección de características muertas, conteo de configuraciones, etc.) sin necesidad de instalar software local. Esta interfaz gráfica reduce significativamente la barrera de entrada para usuarios no técnicos.
- API REST: Flamapy proporciona una API RESTful que expone sus capacidades de análisis como servicios web, permitiendo que otras aplicaciones (como el backend propuesto en esta investigación) integren funcionalidades avanzadas de validación y configuración de modelos de características mediante llamadas HTTP (ibíd.). La API puede desplegarse localmente mediante Docker o accederse a versiones alojadas, facilitando su adopción en arquitecturas de microservicios.
- Línea de comandos y biblioteca Python: para integraciones más profundas, Flamapy ofrece una interfaz de línea de comandos y una biblioteca Python que permite incorporar directamente sus capacidades en aplicaciones desarrolladas en este lenguaje (Team, 2025b).

1.3.3. Oportunidades de Mercado Identificadas

Del análisis realizado se evidencia que el mercado asociado a la temática padece de un vacío crítico en herramientas capaces de integrar la gestión de variabilidad, propia de las SPL, con la planificación curricular educativa, creando una oportunidad para sistemas especializados (Sánchez; Gutiérrez y Cabrera, 2020). Las herramientas existentes de SPL están diseñadas para ingenieros de software, no para profesionales de la educación ni mucho menos para la gestión curricular, limitando su adopción en instituciones educativas y obliga a comprender los conceptos fundamentales de SPLE (Chen; Babar y Ali, 2020). En el caso de Flamapy, destacar que no permite gestionar y operar el FM desde la vista gráfica (visualización del árbol), sino mediante la escritura de UVL. En este escenario, la necesidad de proponer una solución propia no surge de la ausencia total de tecnologías, sino de la falta de una alternativa que articule las fortalezas identificadas en ambos tipos de herramientas y las adapte al dominio educativo.

De los sistemas analizados se reconoce como indispensable la capacidad de representar explícitamente

características con relaciones jerárquicas y restricciones; la validación automática de configuraciones; la posibilidad de derivar múltiples variantes a partir de un modelo común; y la interoperabilidad con ecosistemas académicos existentes. Flamapy, en particular, se destaca su funcionamiento en capturar y modelar los FM a partir de escribir código en UVL, su amplio abanico de exportación a diferentes formatos de archivos y la capacidad de diagnosticar los FM (funcionamiento no ausente en los otros 2 sistemas analizados) en búsquedas de estadísticas, como cantidad de configuraciones, características muertas, etc. La relevancia de Flamapy para la presente investigación es múltiple:

- Su naturaleza como *framework* Python facilita la integración con la propuesta solución, permitiendo reutilizar sus capacidades de análisis en lugar de implementarlas desde cero.
- La existencia de una API REST validada y documentada proporciona un referente arquitectónico para el diseño del servicio.
- Su enfoque en accesibilidad (interfaz web, [Interfaz de Programación de Aplicaciones \(API\)](#)) lo alinea con la necesidad de acercar las técnicas de SPL a gestores académicos no especializados.
- El equipo de desarrollo (Diverso Lab) es el mismo que lidera el estándar UVL, garantizando coherencia con las notaciones adoptadas en esta investigación.

La solución propuesta se posiciona estratégicamente en el mercado educativo como un “Motor de Variabilidad Curricular” especializado, que capitaliza las tendencias actuales de crecimiento en educación híbrida y multimodal, así como las demandas crecientes de personalización educativa y eficiencia en la gestión académica post-pandemia. Su propuesta de valor se sustenta en fortalezas técnicas diferenciadoras, ofreciendo un servidor de aplicaciones (*backend*) que permita diseñar y gestionar modelos de características aplicados a planes de estudio, y derivar itinerarios formativos dinámicos y personalizables. Su valor diferencial radica en su especialización en gestión de variabilidad curricular, incorporar motores de reglas pedagógicas que comprenden prerrequisitos, competencias y secuencias, una arquitectura de API para integración con sistemas existentes y un modelo de datos optimizado para planificación académica. Estas capacidades se traducen en ventajas competitivas tangibles como la automatización de validaciones que reduce errores en diseño curricular, la generación de múltiples variantes de planes de estudio desde un tronco común y su adaptabilidad a contextos institucionales diversos, posicionándose como complemento esencial -no competencia- de los sistemas LMS existentes, al enfocarse específicamente en el diseño y planificación académica más que en la ejecución curricular. En consecuencia, la propuesta investigativa no se limita a replicar herramientas existentes, sino que se posiciona como un puente necesario entre el rigor formal de las SPL y la flexibilidad curricular demandada por la educación contemporánea, constituyendo así una innovación con pertinencia tecnológica y académico-formativa.

1.4. Herramientas y tecnologías

1.5. Metodología de desarrollo del software

Las metodologías de desarrollo de software son enfoques estructurados que guían las distintas fases del ciclo de vida de un proyecto, desde la planificación y análisis hasta la implementación y mantenimiento. Estas metodologías establecen principios, prácticas y procedimientos para gestionar el desarrollo de manera organizada y eficiente, promoviendo la colaboración en equipo y la calidad del producto final (Pacienza, 2015).

A lo largo de esta sección se realiza un análisis de las metodologías de software disponibles y, finalmente, se detallan los principios, fases y prácticas de la metodología seleccionada para guiar el diseño e implementación del sistema propuesto. La elección de la metodología es fundamental para asegurar un proceso de desarrollo ágil, eficiente y adaptable a posibles cambios en los requisitos del proyecto. Estas metodologías facilitan además la detección temprana de errores y el ajuste progresivo de funcionalidades, garantizando la calidad final del sistema.

En la actualidad se pueden diferenciar dos grandes grupos de metodologías de desarrollo de software: las ágiles y las tradicionales. A continuación, se explican las características de cada una de ellas.

1.5.1. Metodologías de desarrollo de software tradicionales

Las metodologías de desarrollo de software tradicionales se caracterizan por definir total y rígidamente los requisitos al inicio de los proyectos de ingeniería de software. Los ciclos de desarrollo son poco flexibles y no permiten realizar cambios, al contrario que las metodologías ágiles; lo que ha propiciado el incremento en el uso de las segundas.

La organización del trabajo de las metodologías tradicionales es lineal, es decir, las etapas se suceden una tras otra y no se puede empezar la siguiente sin terminar la anterior. Tampoco se puede volver hacia atrás una vez se ha cambiado de etapa. Estas metodologías, no se adaptan nada bien a los cambios, y el mundo actual cambia constantemente (Awad, 2005). Las principales metodologías tradicionales o clásicas son:

- **Modelo en Cascada:** también conocido como el ciclo de vida clásico del software, es una de las metodologías de desarrollo de software más antiguas y ampliamente utilizadas. Este enfoque se basa en la idea de que el desarrollo de software debe seguir una secuencia lineal y predefinida de etapas de riguroso orden, cada una de las cuales debe completarse antes de pasar a la siguiente. Los requisitos y especificaciones iniciales no están predispuestos para cambiarse.
- **Incremental:** en esta metodología de desarrollo de software se va construyendo el producto final de manera progresiva. En cada etapa incremental se agrega una nueva funcionalidad, lo que permite ver resultados de una forma más rápida en comparación con el modelo en cascada. El software se puede empezar a utilizar incluso antes de que se complete totalmente y, en general, es mucho más flexible que las demás metodologías.

- **Espiral:** es una combinación de los dos modelos anteriores, que añade el concepto de análisis de riesgo. Se divide en cuatro etapas: planificación, análisis de riesgo, desarrollo de prototipo y evaluación del cliente. El nombre de esta metodología da nombre a su funcionamiento, ya que se van procesando las etapas en forma de espiral. Cuanto más cerca del centro se está, más avanzado está el proyecto.

1.5.2. Metodologías de desarrollo de software ágiles

Hoy en día las metodologías ágiles de desarrollo de software son las más utilizadas debido a su alta flexibilidad y agilidad. Los equipos de trabajo que las utilizan son mucho más productivos y eficientes, ya que saben lo que tienen que hacer en cada momento. Además, la metodología permite adaptar el software a las necesidades que van surgiendo por el camino, lo que facilita construir aplicaciones más funcionales.

Las metodologías ágiles se basan en la metodología incremental, en la que en cada ciclo de desarrollo se van agregando nuevas funcionalidades a la aplicación final. Sin embargo, los ciclos son mucho más cortos y rápidos, por lo que se van agregando pequeñas funcionalidades en lugar de grandes cambios.

Este tipo de metodologías permite construir equipos de trabajo autosuficientes e independientes que se reúnen cada poco tiempo para poner en común las novedades. Poco a poco, se va construyendo y puliendo el producto final, a la vez que el cliente puede ir aportando nuevos requerimientos o correcciones, ya que puede comprobar cómo avanza el proyecto en tiempo real (Pacienza, 2015). Las principales metodologías ágiles son:

- **Kanban:** metodología de trabajo inventada por la empresa de automóviles Toyota. Consiste en dividir las tareas en porciones mínimas y organizarlas en un tablero de trabajo dividido en tareas pendientes, en curso y finalizadas. De esta forma, se crea un flujo de trabajo muy visual basado en tareas prioritarias e incrementando el valor del producto.
- **Scrum:** es también una metodología incremental que divide los requisitos y tareas de forma similar a Kanban. Se itera sobre bloques de tiempos cortos y fijos (entre dos y cuatro semanas) para conseguir un resultado completo en cada iteración. Las etapas son: planificación de la iteración (*planning sprint*), ejecución (*sprint*), reunión diaria (*daily meeting*) y demostración de resultados (*sprint review*). Cada iteración por estas etapas se denomina también *sprint*.
- **Extreme Programming (XP):** es una metodología de desarrollo de software basada en las relaciones interpersonales, que se consideran la clave del éxito. Su principal objetivo es crear un buen ambiente de trabajo en equipo y que haya un *feedback* constante del cliente. El trabajo se basa en 12 conceptos: diseño sencillo, *testing*, refactorización y codificación con estándares, propiedad colectiva del código, programación en parejas, integración continua, entregas semanales e integridad con el cliente, *cliente in situ*, entregas frecuentes y planificación.

1.5.3. Elección de la metodología

En las metodologías tradicionales el principal problema es que nunca se logra planificar bien el esfuerzo requerido para seguir la metodología. Pero entonces, si se logran definir métricas que apoyen la estimación de las actividades de desarrollo, muchas prácticas de metodologías tradicionales podrían ser apropiadas. El no poder predecir siempre los resultados de cada proceso no significa que se esté frente a una disciplina de azar. Lo que significa es que se está frente a la necesidad de adaptación de los procesos de desarrollo que son llevados por parte de los equipos que desarrollan software (Awad, 2005).

Tener metodologías diferentes para aplicar de acuerdo con el proyecto que se desarrolle resulta una idea interesante. Estas metodologías pueden involucrar prácticas tanto de metodologías ágiles como de metodologías tradicionales. De esta manera se podría tener una metodología por cada proyecto, la problemática sería definir cada una de las prácticas, y en el momento preciso definir parámetros para saber cuál usar. Es importante tener en cuenta que el uso de un método ágil no vale para cualquier proyecto. Sin embargo, una de las principales ventajas de los métodos ágiles es su peso inicialmente ligero y por eso las personas que no estén acostumbradas a seguir procesos encuentran estas metodologías bastante agradables.

La tabla 1.3 explica de forma resumida las diferencias que existen entre las metodologías ágiles y las tradicionales:

Cuadro 1.3. Comparación, Metodología Ágil VS Metodología Tradicional (ibíd.)

Parámetros	Metodologías Ágiles	Metodologías Tradicionales
Acercamiento	Adaptivo	Predictivo
Medición del objetivo	Valor para el negocio	Conformación del plan
Tamaño del proyecto	Pequeño	Largo
Tipo de administración	Descentralizado	Autocrático
Perspectiva de cambios	Adaptable a cambio	Sostenible al cambio
Cultura del equipo	Liderazgo\Colaboración	Comandada\Controlada
Documentación	Baja	Pesada
Orientada	Personas	Procesos
Ciclos de desarrollo	Numerosos	Limitados
Dominio de desarrollo	Impredecibles	Predecibles
Planificación inicial	Mínima	Comprensiva
Retorno de la inversión	Temprano en el proyecto	Al final del proyecto
Tamaño del equipo	Pequeño	Grande

Para la selección de la metodología a utilizar se toma como referencia la tabla 1.3. Para ello se analizan las características del proyecto atendiendo a los diferentes aspectos y se selecciona la metodología con la que haya más coincidencias, dando como resultado de este análisis la tabla 1.4.

Cuadro 1.4. Comparación de la Metodología Ágil y la Metodología Tradicional con respecto al proyecto. Fuente: Elaboración propia

Parámetros	Ágiles	Pesadas
Acercamiento	X	
Medición del objetivo	X	
Tamaño del proyecto	X	
Tipo de administración	X	
Perspectiva de cambios	X	
Cultura del equipo	X	
Documentación	X	
Orientada	X	
Ciclos de desarrollo	X	
Dominio de desarrollo	X	
Planificación inicial	X	
Retorno de la inversión		X
Tamaño del equipo	X	
Total:	12	1

Como resultado se obtiene que lo mejor es utilizar una metodología ágil. Ya que, según el análisis realizado, se acomoda mejor a las necesidades y características del proceso a llevar a cabo. Para decidir qué metodología ágil se utilizará, se realizó una comparación entre algunas de las metodologías ágiles más comúnmente utilizadas, la cual se refleja en la tabla 1.5.

Cuadro 1.5. Comparación de Metodología Ágiles. Fuente: Elaboración propia

Parámetros	XP	Scrum
Duración	1 a 2 semanas	2 semanas a 1 mes
Cambios	Sensibles al cambio	No permite cambios
Prioridad	Estricta, priorizada por el cliente	Determinada por el equipo
Prácticas de ingeniería	Prescribe prácticas (pruebas, programación en parejas)	No prescribe prácticas
Tamaño de los proyectos	Pequeños y medianos	Pequeños, medianos y grandes
Tamaño de equipo	Menor que 10	Múltiples equipos menores que 10

Teniendo en cuenta la tabla 1.5 se decide utilizar la metodología XP. Esto se debe a que es muy eficiente durante el proceso de pruebas y planificación, su tasa de error es muy pequeña, facilita los cambios, origina una programación muy organizada, además de fomentar la comunicación entre los desarrolladores y los clientes. También se puede aplicar a cualquier lenguaje de programación, el cliente tiene control sobre las prioridades, durante todo el proyecto se pueden realizar diversas pruebas y, sobre todo, permite ahorrar mucho tiempo y dinero.

1.5.4. Programación Extrema

XP es una metodología ágil de desarrollo de software con bases en la comunicación constante y la retroalimentación. Uno de sus fines principales es el de construir un producto que vaya en línea con los requerimientos del cliente. En ese sentido es adaptable a los cambios, generando una rápida respuesta frente a cualquier inconveniente. Por otro lado, el equipo de trabajo tiene la ventaja de potenciar sus relaciones, ya que el proceso que de este se desprende es abierto, conjunto y de aprendizaje continuo (Pressman, 2014).

Las 4 etapas que sigue **XP** son:

- Planificación: toma como referencia la identificación de las **Historias de Usuario (HU)** con pequeñas versiones que se irán revisando en periodos cortos con el fin de obtener un software funcional.
- Diseño: trabaja el código orientado a objetivos y, sobre todo, usando los recursos necesarios para que funcione.
- Codificación: se refiere al proceso de programación organizada en parejas, estandarizada y que resulte en un código universal entendible.
- Pruebas: consiste en un testeo automático y continuo en el que el cliente tiene voz para validar y proponer.

Los principios de **XP** comprenden diez buenas prácticas que involucran al equipo de trabajo, los procesos y el cliente (*ibíd.*); los cuales son:

- Planificación incremental: Se toman los requerimientos en **HU**, las cuales son negociadas progresivamente con el cliente.
- Entregas pequeñas: Se desarrolla primero la más mínima parte útil que le proporcione funcionalidad al sistema, y poco a poco se efectúan incrementos que añaden funcionalidad a la primera entrega, cada ciclo termina con una entrega del sistema.
- Diseño sencillo: Solo se efectúa el diseño necesario para cumplir con los requerimientos actuales, es decir, no se abordan requerimientos futuros.
- Desarrollo previamente aprobado: Una de las características relevantes y propias de **XP** es que primero se escriben las pruebas y luego se da la codificación, esto con la finalidad de asegurar la satisfacción del requerimiento.
- Limpieza del código o refactorización: Consiste en simplificar y optimizar el programa sin perder funcionalidad, es decir, alterar su estructura interna sin afectar su comportamiento externo.
- Programación en parejas: Es otra de las características de esta metodología, que propone que los desarrolladores trabajen en parejas en una terminal, verificando cada uno el trabajo del otro y ayudándose para buscar las mejores soluciones. Se entiende que de esta forma el trabajo será más eficiente y de mayor calidad.
- Propiedad colectiva: El conocimiento y la información deben ser de todos, por lo tanto no se desarrollan islas de conocimiento, todos los programadores poseen todo el código y cualquiera puede sugerir y realizar mejoras.

- Integración continua: Al terminar una tarea, esta se integra al sistema entero y se realizan pruebas de unidad a todo el sistema, esta práctica permite que la aplicación sea más funcional en cada iteración y garantiza su funcionamiento con los demás módulos del sistema.
- Ritmo sostenible: No es aceptable trabajar durante grandes cantidades de horas ya que se considera que puede reducir la calidad del código y la productividad del equipo a mediano plazo, se sugieren 40 horas semanales.
- Cliente presente: Se debe tener un representante (cliente o usuario final) a tiempo completo, ya que en **XP** este hace parte del equipo de desarrollo y es responsable de formular los requerimientos para el desarrollo del sistema.

Artefactos

Las metodologías utilizan artefactos para describir las actividades o procedimientos que se deben seguir durante su desarrollo. A continuación, se describen los artefactos de **XP**:

- **HU**: Representan una breve descripción del comportamiento del sistema. Emplea terminología del cliente sin lenguaje técnico. Se realiza una por cada característica principal del sistema. Se emplean para hacer estimaciones de tiempo y para el plan de lanzamientos. Reemplazan un gran documento de requisitos y presiden la creación de las pruebas de aceptación. Estas deben proporcionar sólo el detalle suficiente como para poder hacer razonable la estimación de cuánto tiempo requiere la implementación de la **HU**, difiere de los casos de uso porque son escritos por el cliente, no por los programadores, empleando terminología del cliente (Pressman, 2014).
- **Tareas de Ingeniería**: Una **HU** se descompone en varias tareas de ingeniería, que describen las actividades que se realizarán en cada una de ellas. Así mismo, las tareas de ingeniería se vinculan más al desarrollador, ya que permite tener un acercamiento con el código (*ibíd.*).
- **Tarjetas Clase, Responsabilidad y Colaboración (CRC)**: Estas tarjetas se dividen en tres secciones que contienen la información del nombre de la clase, sus responsabilidades y sus colaboradores. Una clase es cualquier persona, cosa, evento, concepto, pantalla o reporte. Las responsabilidades de una clase son las cosas que conoce y las que realiza, sus atributos y métodos. Los colaboradores son las demás clases con las que trabaja en conjunto para llevar a cabo sus responsabilidades (*ibíd.*).

En síntesis, actualmente **XP** es una de las metodologías de mayor aceptación en la industria del software. Su enfoque basado en los métodos ágiles, su énfasis en la gestión del recurso humano, el cual es uno de los puntos más críticos en todo proyecto, y sus principios de previsibilidad y adaptabilidad hacen de esta metodología una buena opción a seguir.

1.6. Conclusiones del capítulo

Posicionamiento de la presente investigación

La presente investigación se inscribe precisamente en este punto de intersección: se propone desarrollar un servicio backend que, basado en los principios de [SPL](#) y utilizando [UWL](#) como lenguaje de modelado, permita a los gestores académicos representar y gestionar la variabilidad curricular de forma sistemática. Este servicio constituye la base computacional sobre la cual podrán construirse, en trabajos futuros, configuradores curriculares intuitivos que acerquen estas potentes técnicas a la comunidad educativa, democratizando así el acceso a los beneficios de la ingeniería de líneas de productos en el dominio de la educación superior.

2.1. Sección de prueba

Escribir aquí el capítulo 2. Prueba de glosario [engine](#). A continuación un ejemplo de enlace al anexo [Bloque Corona](#). Este es un ejemplo de matriz con bordes numerados (Mathew y Rao, 2010).

<i>Padre</i>	1	2	3	4	
1	0	1	0	0	→ Seleccionada aleatoriamente
2	0	0	0	1	
3	1	0	0	0	→ Seleccionada aleatoriamente
4	0	0	1	0	

<i>Madre</i>	1	2	3	4
1	1	0	0	0
2	0	1	0	0
3	0	0	1	0
4	0	0	0	1

$$\begin{array}{c}
 \text{Hijo} \\
 \left[\begin{array}{ccccc}
 & 1 & 2 & 3 & 4 \\
 1 & 0 & 1 & 0 & 0 \\
 2 & 0 & 0 & 1 & 0 \\
 3 & 1 & 0 & 0 & 0 \\
 4 & 0 & 0 & 0 & 1
 \end{array} \right]
 \end{array} \quad (2.1.1)$$

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Este es un ejemplo de cita (Mathew y Rao, 2010) Esta es una prueba de cómo se escribe un pseudocódigo en $\text{\LaTeX}$.

2.2. Ejemplos de código fuente

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna.

Algoritmo 1 Estrategia de BellmanKalaba

```

1: procedure BELLMANKALABA( $G, u, l, p$ )
2:   for all  $v \in V(G)$  do
3:      $l(v) \leftarrow \infty$ 
4:   end for
5:    $l(u) \leftarrow 0$ 
6:   repeat
7:     for  $i \leftarrow 1, n$  do
8:        $min \leftarrow l(v_i)$ 
9:       for  $j \leftarrow 1, n$  do
10:        if  $min > e(v_i, v_j) + l(v_j)$  then
11:           $min \leftarrow e(v_i, v_j) + l(v_j)$ 
12:           $p(i) \leftarrow v_j$ 
13:        end if
14:      end for
15:       $l'(i) \leftarrow min$ 
16:    end for
17:     $changed \leftarrow l \neq l'$ 
18:     $l \leftarrow l'$ 
19:  until  $\neg changed$ 
20: end procedure

```

Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetur adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula. Este es un ejemplo de cita (Mathew y Rao, 2010)

Código fuente 2.1. Búsqueda de máximo

```

int maxSearch( const int* v )
{
    /**
     * This is a text that takes
     * two lines
     */
    int currentMax = 0;
    for(int i = 0; i < MAX_NUMBER; i++)
    {
        currentMax = max(v[i], currentMax); // here we are using std::max
    }
}

```

```

    }

    return currentMax;
}

```

Código fuente 2.2. Ejemplo de código en Java

```

1  /**
2   * HelloWorldApp class prints
3   * "Hello World!" to standard output
4   */
5  public class HelloWorldApp{
6      public static void main(String argv[])
7      {
8          // Display string
9          System.out.println("Hello World!");
10     }
11 }

```

Código fuente 2.3. Ejemplo de código en PHP

```

1  <?php
2      /**
3       * Retrieves a list of models based on the current search/filter conditions.
4       * @return CActiveDataProvider the data provider that can return the models
5         based on the search/filter conditions.
6       */
7  public function search()
8  {
9      // Warning: Please modify the following code to remove attributes that
10     // should not be searched.
11
12     $criteria=new CDbCriteria;
13     $criteria->compare('idtarea',$this->idtarea);
14     $criteria->compare('nombre',$this->nombre,true);
15     $criteria->compare('descripcion',$this->descripcion,true);
16     $criteria->compare('fecha',$this->fecha,true);
17     $criteria->compare('idinstructora',$this->idinstructora);
18
19     //$user=Yii::app()->user->name;
20     $rol=Usser::model()->getRolThisUser($user);
21     $rol=Usser::model()->getNameRol($rol);
22     if('Instructora'==$rol)
23     {
24         $criteria->join="JOIN instructora ON(t.idinstructora=instructora.
25         idinstructora ) ";
26         $criteria->condition="instructora.idusuario='".$user."'";
27     }
28
29     return new CActiveDataProvider($this, array(
30         'criteria'=>$criteria,
31         'pagination'=>array('pageSize'=>5)
32     ));
33 }

```

```

30     ));
31 }
32 ?>

```

Código fuente 2.4. Ejemplo de código en PHP

```

1 public function evidenciaXUsuarioAction($id)
2 {
3     $em = $this->getDoctrine()->getManager();
4
5     $usuario = $em->getRepository('GIVIPBundle:Usuario')->find($id);
6     if(!$usuario){
7         $this->get('session')->getFlashBag()->add('error', 'El usuario
            seleccionado no existe en el sistema.');
```

Borrador

```

8
9         return $this->redirect($this->generateUrl('evidencia'));
10    }
11    $nombre = $usuario->getNombre() ." ". $usuario->getApellidos();
12    if($this->get('security.context')->isGranted('ROLE_VIP1')){
13        $consulta = $em->createQuery("select e, i, u from GIVIPBundle:Evidencia e
            JOIN e.indicador i JOIN e.usuario u WHERE e.
            usuario=:id");
14
15    }
16    else
17    {
18        $consulta = $em->createQuery("select e, i, u from GIVIPBundle:Evidencia e
            JOIN e.indicador i JOIN e.usuario u WHERE e.usuario=:id and e.estado=
            true");
19    }
20
21    $consulta->setParameter('id', $id);
22
23    $res = $consulta->getResult();
24    $paginator = $this->get('ideup.simple_paginator');
25    $paginator->setItemsPerPage(10, 'resultado');
26    $entities = $paginator->paginate($res, 'resultado')->getResult();
27
28    $deleteForm = $this->createDeleteForm(0);
29    $buscarForm = $this->createSearchForm();
30
31    return $this->render('GIVIPBundle:Evidencia:evidenciausuario.html.twig',
        array(
32        'entities' => $entities, 'delete_form' => $deleteForm->createView(),
33        'buscar_form' => $buscarForm->createView(), 'usuario'=>$nombre
34    ));
35 }
```

Código fuente 2.5. Ejemplo de código en Python

```

1 class TelegramRequestHandler(object):
2     def handle(self):
3         addr = self.client_address[0]           # Client IP-adress
4         telegram = self.request.recv(1024)      # Recieve telegram
5         print "From: %s, Received: %s" % (addr, telegram)
6         return

```

Código fuente 2.6. Ejemplo de código en htmlcssjs

```
1 @media only screen and (min-width: 768px) and (max-width: 991px) {
2
3   #main {
4     width: 712px;
5     padding: 100px 28px 120px;
6   }
7
8   /* .mono {
9     font-size: 90%;
10  } */
11
12  .cssbtn a {
13    margin-top: 10px;
14    margin-bottom: 10px;
15    width: 60px;
16    height: 60px;
17    font-size: 28px;
18    line-height: 62px;
19  }
```

Código fuente 2.7. Ejemplo de código en htmlcssjs

```
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <title>Listings Style Test</title>
5     <meta charset="UTF-8">
6     <style>
7       /* CSS Test */
8       * {
9         padding: 0;
10        border: 0;
11        margin: 0;
12      }
13    </style>
14    <link rel="stylesheet" href="css/style.css" />
15  </head>
16  <header> hey </header>
17  <article> this is a article </article>
18  <body>
19    <!-- Paragraphs are fine -->
20    <div id="box">
21      <p>
22        Hello World
23      </p>
24      <p>Hello World</p>
25      <p id="test">Hello World</p>
26      <p></p>
27    </div>
28    <div>Test</div>
29    <!-- HTML script is not consistent -->
30    <script src="js/benchmark.js"></script>
31    <script>
32      function createSquare(x, y) {
```

```
33     // This is a comment.
34     var square = document.createElement('div');
35     square.style.width = square.style.height = '50px';
36     square.style.backgroundColor = 'blue';
37
38     /*
39      * This is another comment.
40      */
41     square.style.position = 'absolute';
42     square.style.left = x + 'px';
43     square.style.top = y + 'px';
44
45     var body = document.getElementsByTagName('body')[0];
46     body.appendChild(square);
47 };
48
49 // Please take a look at +=
50 window.addEventListener('mousedown', function(event) {
51     // German umlaut test: Beruehrungspunkt ermitteln
52     var x = event.touches[0].pageX;
53     var y = event.touches[0].pageY;
54     var lookAtThis += 1;
55 });
56 </script>
57 </body>
58 </html>
```

Ejemplo de tabla grande

3.1. Tabla aleatoria

Cuadro 3.1. Feasible triples for highly variable Grid, MLMMH.

Time (s)	Triple chosen	Other feasible triples
0	(1, 11, 13725)	(1, 12, 10980), (1, 13, 8235), (2, 2, 0), (3, 1, 0)
2745	(1, 12, 10980)	(1, 13, 8235), (2, 2, 0), (2, 3, 0), (3, 1, 0)
5490	(1, 12, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
8235	(1, 12, 16470)	(1, 13, 13725), (2, 2, 2745), (2, 3, 0), (3, 1, 0)
10980	(1, 12, 16470)	(1, 13, 13725), (2, 2, 2745), (2, 3, 0), (3, 1, 0)
13725	(1, 12, 16470)	(1, 13, 13725), (2, 2, 2745), (2, 3, 0), (3, 1, 0)
16470	(1, 13, 16470)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
19215	(1, 12, 16470)	(1, 13, 13725), (2, 2, 2745), (2, 3, 0), (3, 1, 0)
21960	(1, 12, 16470)	(1, 13, 13725), (2, 2, 2745), (2, 3, 0), (3, 1, 0)
24705	(1, 12, 16470)	(1, 13, 13725), (2, 2, 2745), (2, 3, 0), (3, 1, 0)
27450	(1, 12, 16470)	(1, 13, 13725), (2, 2, 2745), (2, 3, 0), (3, 1, 0)
30195	(2, 2, 2745)	(2, 3, 0), (3, 1, 0)
32940	(1, 13, 16470)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
35685	(1, 13, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
38430	(1, 13, 10980)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
41175	(1, 12, 13725)	(1, 13, 10980), (2, 2, 2745), (2, 3, 0), (3, 1, 0)
43920	(1, 13, 10980)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
46665	(2, 2, 2745)	(2, 3, 0), (3, 1, 0)
49410	(2, 2, 2745)	(2, 3, 0), (3, 1, 0)
52155	(1, 12, 16470)	(1, 13, 13725), (2, 2, 2745), (2, 3, 0), (3, 1, 0)
54900	(1, 13, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
57645	(1, 13, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
60390	(1, 12, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
63135	(1, 13, 16470)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
65880	(1, 13, 16470)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
68625	(2, 2, 2745)	(2, 3, 0), (3, 1, 0)
71370	(1, 13, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
74115	(1, 12, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
76860	(1, 13, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
79605	(1, 13, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)

Continued on next page

Cuadro 3.1. continued from previous page

Time (s)	Triple chosen	Other feasible triples
82350	(1, 12, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
85095	(1, 12, 13725)	(1, 13, 10980), (2, 2, 2745), (2, 3, 0), (3, 1, 0)
87840	(1, 13, 16470)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
90585	(1, 13, 16470)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
93330	(1, 13, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
96075	(1, 13, 16470)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
98820	(1, 13, 16470)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
101565	(1, 13, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
104310	(1, 13, 16470)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
107055	(1, 13, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
109800	(1, 13, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
112545	(1, 12, 16470)	(1, 13, 13725), (2, 2, 2745), (2, 3, 0), (3, 1, 0)
115290	(1, 13, 16470)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
118035	(1, 13, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
120780	(1, 13, 16470)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
123525	(1, 13, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
126270	(1, 12, 16470)	(1, 13, 13725), (2, 2, 2745), (2, 3, 0), (3, 1, 0)
129015	(2, 2, 2745)	(2, 3, 0), (3, 1, 0)
131760	(2, 2, 2745)	(2, 3, 0), (3, 1, 0)

3.2. Tablas de ingeniería

Cuadro 3.2. Historia de usuario # 1

Historia de usuario	
Número: 1	Nombre: Cargar archivo
Usuario: Especialista	
Prioridad en negocio: Alta	Riesgo en desarrollo: Bajo
Puntos estimados: 0.8	Iteración asignada: 1
Programador responsable: Ernesto Gil Capote	
Descripción: Permite cargar los datos de los casos de estudio a utilizar en el módulo con el objetivo de determinar secuencias factibles. Estos están almacenados en archivos con extensión .asp. La estructura está compuesta por los parámetros de entrada de la aplicación para cada caso de estudio tales como: dimensión, nombre de la estructura mecánica, cantidad de piezas, la MD correspondiente y el vector de herramientas en caso de que lo posea. • rojo • blanco	
Observaciones: En caso de que el usuario no cargue un archivo, el sistema debe mostrar un mensaje de alerta con dicha explicación. Si se introduce un archivo que contiene caracteres no válidos o falta alguno de los elementos necesarios para la ejecución del módulo, este debe lanzar la excepción correspondiente. En caso de que el usuario no cargue un archivo, el sistema debe mostrar un mensaje de alerta con dicha explicación. Si se introduce un archivo que contiene caracteres no válidos o falta alguno de los elementos necesarios para la ejecución del módulo, este debe lanzar la excepción correspondiente.	

Esta es referencia a una tabla de historia de usuario [3.2](#)

Cuadro 3.3. Tarjeta CRC # 1

Tarjeta CRC	
Clase: AlgoritmoGenetico	
Responsabilidad	Colaboración
• Crear el conjunto de genes necesarios para la creación de un individuo • Realizar el proceso de mutación y recombinación • Crear el conjunto de genes necesarios para la creación de un individuo • Realizar el proceso de mutación y recombinación	Población OperadorProbabilidad OperadorParejas OperadorReproduccion Población

Cuadro 3.4. Tarjeta CRC # 2

Tarjeta CRC	
Clase: AlgoritmoGenetico	
Responsabilidad	Colaboración
• Crear el conjunto de genes necesarios para la creación de un individuo • Realizar el proceso de mutación y recombinación • Crear el conjunto de genes necesarios para la creación de un individuo • Realizar el proceso de mutación y recombinación	Población OperadorProbabilidad OperadorParejas OperadorReproduccion Población

Cuadro 3.5. Estimación de esfuerzo por historia de usuario

Iteración	Historias de usuario		Puntos estimados (semanas)
1	1	Cargar Archivo	0.8
	2	Determinar Secuencia de Ensamble	0.2
	3	Gestionar restricción cambios de herramienta	0.5
	4	Graficar secuencias de ensamble	0.6
2	5	Graficar secuencias de ensamble	0.3
	6	Graficar secuencias de ensamble	0.3
	7	Graficar secuencias de ensamble	0.3
	8	Graficar secuencias de ensamble	0.2
	9	Graficar secuencias de ensamble	0.2
3	10	Determinar Secuencia de Ensamble	0.5
4	11	Graficar secuencias de ensamble	0.1
	12	Graficar ensamble	0.1
Total			4.1

Cuadro 3.6. Estimación de esfuerzo por historia de usuario

Iteración	Historias de usuario		Puntos estimados (semanas)
1	1	Cargar Fichero	0.8
	2	Determinar Secuencia de Ensamble	0.2
	3	Gestionar restricción cambios de herramienta	0.5
	4	Esta es la prueba para el tinguiri	0.5
2	5	Graficar secuencias de ensamble	0.3
	6	Graficar secuencias de ensamble	0.3
	7	Graficar secuencias de ensamble	0.2
	8	Graficar secuencias de ensamble	0.2
3	9	Determinar Secuencia de Ensamble	2.5
4	10	Graficar secuencias de ensamble	1.1
Total			6.6

Esto es una prueba de Tabla 3.5

Cuadro 3.7. Plan de duración de las iteraciones

Iteración	Historias de usuario		Duración (semanas)
1	1	Cargar Fichero	3
	2	Determinar Secuencia de Ensamble	
	3	Gestionar restricción cambios de herramienta	
	4	Esta es la prueba para el tinguiri	
2	5	Graficar secuencias de ensamble	1.0
	6	Graficar secuencias de ensamble	
	7	Graficar secuencias de ensamble	
	8	Graficar secuencias de ensamble	
3	9	Determinar Secuencia de Ensamble	2.5
4	10	Graficar secuencias de ensamble	1.1
Total			7.6

Esto es una prueba de Tabla 3.7

Cuadro 3.8. Tarea de ingeniería # 1

Tarea	
Número de tarea: 1	Número de Historia de usuario: 4
Nombre de la tarea: Registrar usuario	
Tipo de tarea: Registrarse	Puntos estimados: 1
Fecha de inicio: 6 de mayo de 2014	Fecha de fin: 24 de octubre de 2014
Descripción: El usuario introduce sus datos para poder registrarse.	

Cuadro 3.9. Tarea de desarrollo # 1

Tarea	
Número de tarea: 1	Número de Historia de usuario: 4
Nombre de la tarea: Registrar usuario	
Tipo de tarea: Registrarse	Puntos estimados: 1

Continúa en la próxima página

Cuadro 3.9. Continuación de la página anterior

Fecha de inicio: 6 de mayo de 2014	Fecha de fin: 24 de octubre de 2014
Programador responsable: Alberto Medina Ramírez	
Descripción: El usuario introduce sus datos para poder registrarse.	

Cuadro 3.10. Tarea de desarrollo # 2

Tarea	
Número de tarea: 2	Número de Historia de usuario: 4
Nombre de la tarea: Registrar usuario	
Tipo de tarea: Registrarse	Puntos estimados: 1
Fecha de inicio: 6 de mayo de 2014	Fecha de fin: 24 de octubre de 2014
Descripción: El usuario introduce sus datos para poder registrarse.	

Cuadro 3.11. Tarea de ingeniería # 2

Tarea	
Número de tarea: 2	Número de Historia de usuario: 4
Nombre de la tarea: Registrar usuario	
Tipo de tarea: Registrarse	Puntos estimados: 1
Fecha de inicio: 6 de enero de 2014	Fecha de fin: 24 de noviembre de 2014
Programador responsable: Alberto Medina Ramírez	
Descripción: El usuario introduce sus datos para poder registrarse.	

Referencia 3.9

Cuadro 3.12. Prueba de aceptación # 1

Caso de prueba de aceptación	
Código: HU2_P1	Historia de usuario: 2
Nombre: Autenticar usuario en el sistema.	
Descripción: Prueba para la funcionalidad autenticar usuario.	
Condiciones de ejecución: El usuario debe estar previamente registrado. El usuario y contraseña deben ser válidos.	
Pasos de ejecución: Se intenta autenticar un usuario en el sistema con los datos válidos.	
Resultados esperados: El usuario se autentifica correctamente en el sistema.	

Cuadro 3.13. Prueba de aceptación # 2

Caso de prueba de aceptación	
Código: HU2_P1	Historia de usuario: 2
Nombre: Autenticar usuario en el sistema.	
Descripción: Prueba para la funcionalidad autenticar usuario.	

Continúa en la próxima página

Cuadro 3.13. Continuación de la página anterior

Condiciones de ejecución: El usuario debe estar previamente registrado. El usuario y contraseña deben ser válidos.
Pasos de ejecución: Se intenta autenticar un usuario en el sistema con los datos válidos.
Resultados esperados: El usuario se autentifica correctamente en el sistema.

Prueba de link [3.12](#)

Escribir aquí el capítulo 3. Otra prueba más [engine](#). Esto es un ejemplo de cómo se puede *linkear* el anexo [Controlador Industrial](#).

Esto es otra cita de la inicial como ejemplo también (Ou y Xu, 2013). Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetur adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.

Fusce mauris. Vestibulum luctus nibh at lectus. Sed bibendum, nulla a faucibus semper, leo velit ultricies tellus, ac venenatis arcu wisi vel nisl. Vestibulum diam. Aliquam pellentesque, augue quis sagittis posuere, turpis lacus congue quam, in hendrerit risus eros eget felis. Maecenas eget erat in sapien mattis porttitor. Vestibulum porttitor. Nulla facilisi. Sed a turpis eu lacus commodo facilisis. Morbi fringilla, wisi in dig-

nissim interdum, justo lectus sagittis dui, et vehicula libero dui cursus dui. Mauris tempor ligula sed lacus. Duis cursus enim ut augue. Cras ac magna. Cras nulla. Nulla egestas. Curabitur a leo. Quisque egestas wisi eget nunc. Nam feugiat lacus vel est. Curabitur consectetur. Otra prueba más [engine](#) Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa. Otra prueba más [engine](#) Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris. Esto es una prueba de link al Anexo [B.2](#). Esto es una prueba de link a la palabra [sentencia](#)

Conclusiones

Escribir aquí las conclusiones.

Borrador

Recomendaciones

Escribe aquí las recomendaciones de tu trabajo.

Borrador

Referencias bibliográficas

- ACADEMIALAB, [s.f.]. *Línea de productos de software*. Url: <https://academia-lab.com/enciclopedia/linea-de-productos-de-software/>. Recuperado 25 de octubre de 2025 (vid. pág. 6).
- ANTHOLOGY INC., 2025. *Blackboard Learn: Plataforma para la enseñanza y el aprendizaje*. Url: <https://www.anthology.com/blackboard-learn>. Recuperado 25 de octubre de 2025 (vid. pág. 27).
- APEL, S.; BATORY, D.; KÄSTNER, C. y SAAKE, G., 2013. *Feature-Oriented Software Product Lines: Concepts and Implementation*. Springer (vid. págs. 7, 17, 26).
- AWAD, Mohamed A, 2005. A comparison between agile and traditional software development methodologies. *University of Western Australia*. Vol. 30, págs. 1-69 (vid. págs. 31, 33).
- BATES, A. W., 2019. *Teaching in a Digital Age: Guidelines for Designing Teaching and Learning*. Tony Bates Associates Ltd (vid. pág. 2).
- BENAVIDES, D.; SEGURA, S. y RUIZ-CORTÉS, A., 2010. Automated analysis of feature models 20 years later: A literature review. *Information Systems*. Vol. 35, n.º 6, págs. 615-636 (vid. págs. 8, 11, 13, 17).
- BENAVIDES, David; SEGURA, Sergio y RUIZ-CORTÉS, Antonio, 2010. Principles of Feature Modeling. En: *Proceedings of the 7th International Conference on Software Product Lines (SPLC)*. Springer, págs. 13-22 (vid. pág. 8).
- BENAVIDES, David; SUNDERMANN, Chico; FEICHTINGER, Kevin; GALINDO, José A.; RABISER, Rick y THÜM, Thomas, 2025. UVL: Feature modelling with the Universal Variability Language. *Journal of Systems and Software*. Disp. desde doi: [10.1016/j.jss.2024.112326](https://doi.org/10.1016/j.jss.2024.112326) (vid. pág. 14).
- BENAVIDES, David; TRINIDAD, José-Antonio y RUIZ-CORTÉS, Antonio, 2005. A taxonomy of variability constraints for feature models. En: *SPLC*. Springer, págs. 99-108 (vid. págs. 10-12).
- CHEN, L.; BABAR, M. A. y ALI, N., 2020. Variability management in software product lines: A systematic review. *ACM Computing Surveys*. Vol. 53, n.º 4, págs. 1-45 (vid. págs. 17, 29).
- CHIMALAKONDA, Sridhar y NORI, Kesav V., 2012. Accelerating educational technologies using software product lines. En: *2012 IEEE International Conference on Technology Enhanced Education (ICTEE)*, págs. 1-4. Disp. desde doi: [10.1109/ICTEE.2012.6208608](https://doi.org/10.1109/ICTEE.2012.6208608) (vid. págs. 18, 19).
- DEL ÁGUILA, L. et al., 2014. La gestión por procesos en las Instituciones de Educación Superior. *UT-Ciencia*. Vol. 1, n.º 3, págs. 140-149. Url: <https://dialnet.unirioja.es/servlet/articulo?codigo=9596104>. [citation:1] (vid. pág. 1).
- DÍAZ, V., 2002. Flexibilidad y educación superior en Colombia. *Instituto Colombiano para el Fomento de la Educación Superior*. Citado en: https://www.mineducacion.gov.co/1621/articles-106706_archivo_pdf.pdf (vid. pág. 18).
- ELLUCIAN COMPANY, 2015. *Banner de Ellucian. Características generales* (vid. pág. 27).

- GALÁN, David y OTROS, 2014. Building Families of Software Products for e-Learning Platforms: A Case Study. *IEEE Revista Iberoamericana de Tecnologías del Aprendizaje*. Vol. 9, n.º 2, págs. 64-71. Disp. desde doi: [10.1109/RITA.2014.2317524](https://doi.org/10.1109/RITA.2014.2317524) (vid. pág. 19).
- GALINDO, José A.; HORCAS, Jose-Miguel; FELFERNING, Alexander; FERNANDEZ-AMOROS, David y BENAVIDES, David, 2023. FLAMA: A collaborative effort to build a new framework for the automated analysis of feature models. En: *Proceedings of the 27th ACM International Systems and Software Product Line Conference (SPLC '23) - Tool Track*. Disp. desde doi: [10.1145/3579028.3609008](https://doi.org/10.1145/3579028.3609008) (vid. pág. 29).
- GEEKSFORGEES, [s.f.]. *What is Feature Engineering?* Url: <https://www.geeksforgeeks.org/machine-learning/what-is-feature-engineering/> (vid. pág. 8).
- GÓMEZ, YA, 2023. Innovación educativa y gestión curricular. *Anales de la Real Academia de Doctores. Obtenido de https://www.rade.es/imageslib/PUBLICACIONES/ARTICULOS/V8N3*. Vol. 2 (vid. págs. 1, 2, 18).
- GUTAMA, Jonnathan Fabricio; ORTIZ GONZÁLEZ, Jhordi Joel; MOSCOSO BERNAL, Santiago Arturo y AYABACA LANDI, David Santiago, 2025. Diseño de gestión de procesos para la Secretaría y Gestores de la Unidad Académica de Ingeniería Industria y Construcción de la Universidad Católica de Cuenca. *South Florida Journal of Development*. Disp. desde doi: [10.46932/sfjdv6n3-004](https://doi.org/10.46932/sfjdv6n3-004). [citation:4] (vid. pág. 1).
- INGENIERÍA DE SOFTWARE, [s.f.]. *Líneas de Producto Software para acelerar el desarrollo de aplicaciones relacionadas*. Url: <https://ingenieriadesoftware.es/lineas-producto-software/>. Recuperado 25 de octubre de 2025 (vid. pág. 6).
- INSTRUCTURE, 2025. *Canvas LMS: Enseñanza y aprendizaje en cualquier lugar*. Url: <https://www.instructure.com/canvas>. Recuperado 25 de octubre de 2025 (vid. pág. 27).
- KANG, Kyo C.; COHEN, Sholom; HESS, James A.; NOVAK, William E. y PETERSON, A. Spencer, 1990. *Feature-Oriented Domain Analysis (FODA) Feasibility Study*. Inf. téc., CMU/SEI-90-TR-21. Software Engineering Institute, Carnegie Mellon University (vid. pág. 8).
- MATHEW, Arun Tom y RAO, C.S.P., 2010. A Novel Method of Using API to Generate Liaison Relationships from an Assembly. *Journal of Software Engineering & Applications*. Vol. 3, n.º 2, págs. 167-175 (vid. págs. 37-39).
- MENDONÇA, M y WASOWSKI, A, 2009. SAT-based analysis of feature models is easy. En: *SPLC*. ACM, págs. 231-240 (vid. pág. 11).
- MOODLE PROJECT, 2025. *¿Qué es Moodle? Visión general de la plataforma*. Url: <https://moodle.com/es/what-is-moodle/>. Recuperado 25 de octubre de 2025 (vid. págs. 27, 28).
- OFICINA DE TRANSFERENCIA DE RESULTADOS DE INVESTIGACIÓN - UNIVERSIDAD DE SEVILLA, 2025. *Investigadores de la US lideran un nuevo lenguaje universal de variabilidad del software* [online]. [visitado 2026-03-05]. Disponible en: <https://www.investigacion.us.es/noticias/investigadores-de-la-us-lideran-un-nuevo-lenguaje-universal-de-variabilidad-del-software> (vid. pág. 14).
- OKADA, Takahiro; HISAZUMI, Kenji; NAKANISHI, Tsuneo y FUKUDA, Akira, 2009. A Methodology for improving reusability of educational materials using product line software engineering. En: *Proceedings of the 17th International Conference on Computers in Education (ICCE 2009)*, págs. 170-172. SCOPUS ID: 84857079693 (vid. págs. 18, 19).

- OU, Li-Ming y XU, Xun, 2013. Relationship matrix based automatic assembly sequence generation from a CAD model. *Computer-Aided Design*. Vol. 45, n.º 7, págs. 1053-1067. ISSN 0010-4485. Disp. desde doi: <http://dx.doi.org/10.1016/j.cad.2013.04.002> (vid. pág. 50).
- PACIENZA, J., 2015. *Metodologías de desarrollo de software*. UCA (vid. págs. 31, 32).
- POHL, K.; BÖCKLE, G. y LINDEN, F. J. van der, 2005. *Software Product Line Engineering: Foundations, Principles and Techniques*. Springer (vid. págs. 7, 8).
- PRESSMAN, R. S., 2014. *Software Engineering: A Practitioner's Approach*. 8.^a ed. McGraw-Hill Education (vid. págs. 35, 36).
- PTC, 2025. *Feature Models and Features: What Are They*. Url: <https://www.ptc.com/en/blogs/alm/modelling-features>. Recuperado 25 de octubre de 2025 (vid. pág. 8).
- PURE-SYSTEMS GMBH, 2024. *pure::variants Product Overview*. Url: <https://www.pure-systems.com/en/products/purevariants>. Recuperado 25 de octubre de 2025 (vid. pág. 28).
- RODRÍGUEZ, P.; VIZCAÍNO, A. y PIATTINI, M., 2019. Herramientas de gestión curricular: Una evaluación de la situación actual. *IEEE Revista Iberoamericana de Tecnologías del Aprendizaje*. Vol. 14, n.º 3, págs. 112-120 (vid. pág. 17).
- SÁNCHEZ, J.; GUTIÉRREZ, F. L. y CABRERA, M., 2020. Hacia una metodología para la gestión de la variabilidad en planes de estudio de ingeniería de software. *Journal of Educational Computing Research*. Vol. 58, n.º 5, págs. 1020-1045 (vid. págs. 2, 17, 27, 29).
- SG BUZZ, [s.f.]. *Líneas de Productos de Software*. Url: <https://sg.com.mx/revista/34/lineas-productos-software>. Recuperado 25 de octubre de 2025 (vid. pág. 5).
- SILVA MARCOLINO, Anderson y FRANCINE BARBOSA, Ellen, 2017. Towards a Software Product Line Architecture to Build M-learning Applications for the Teaching of Programming. En: *Proceedings of the 50th Hawaii International Conference on System Sciences (HICSS 2017)*. Disp. desde doi: [101255/41922](https://doi.org/10.1255/41922) (vid. págs. 18, 19).
- SOLIS PINO, Andrés Felipe; RUIZ, Pablo H. y HURTADO ALEGRIA, Julio Ariel, 2022. A Software Products Line as Educational Tool to Learn Industrial Robots Programming with Arduino. *Electronics*. Vol. 11, n.º 5, pág. 769. Disp. desde doi: [10.3390/electronics11050769](https://doi.org/10.3390/electronics11050769) (vid. pág. 19).
- SOOS, Mate; GOCHT, Stephan y AL., et, 2020. Ganak: A scalable probabilistic model counter. En: *SAT Conference*. Springer (vid. pág. 11).
- SOTO GRANT, Andrea, 2022. La gestión por procesos como herramienta fundamental en el aseguramiento de la calidad de las carreras universitarias. *Revista Electrónica Actualidades Investigativas en Educación*. Vol. 22, n.º 2, págs. 1-24. Disp. desde doi: [10.15517/aie.v22i2.48906](https://doi.org/10.15517/aie.v22i2.48906). [citation:2] (vid. pág. 1).
- SUNDERMANN, C.; DÖRR, L.; JUNKER, M. y KEHRER, T., 2023. Evaluating state-of-the-art #SAT solvers on industrial configuration spaces. *Empirical Software Engineering*. Vol. 28, n.º 29 (vid. págs. 10, 11).
- SUNDERMANN, Chico; FEICHTINGER, Kevin; ENGELHARDT, Dominik; RABISER, Rick y THÜM, Thomas, 2021. Yet another textual variability language? a community effort towards a unified language. En: *Proceedings of the 25th ACM International Systems and Software Product Line Conference (SPLC '21)*. Disp. desde doi: [10.1145/3461001.3471145](https://doi.org/10.1145/3461001.3471145) (vid. pág. 14).

- SUNDERMANN, Chico; VILL, Stefan; THÜM, Thomas; FEICHTINGER, Kevin; AGARWAL, Prankur; RABISER, Rick; GALINDO, José A. y BENAVIDES, David, 2023. UVLParse: Extending UVL with Language Levels and Conversion Strategies. En: *Proceedings of the 27th ACM International Systems and Software Product Line Conference (SPLC '23) - Tool Track*. Disp. desde dor: [10.1145/3579028.3609013](https://doi.org/10.1145/3579028.3609013) (vid. pág. 14).
- TEAM, Flamapy, 2024. *FLAMAPY rest API* [online]. [visitado 2026-03-08]. Disponible en: <https://pypi.org/project/flamapy-rest/> (vid. pág. 29).
- TEAM, Flamapy, 2025a. *FeatureIDE Reader - Flamapy Documentation* [online]. [visitado 2026-03-08]. Disponible en: https://docs.flamapy.org/framework/transformations/featureide_reader (vid. pág. 29).
- TEAM, Flamapy, 2025b. *Flamapy Documentation* [online]. [visitado 2026-03-08]. Disponible en: <https://docs.flamapy.org/> (vid. pág. 29).
- TEAM, Flamapy, 2025c. *FM to BDD - Flamapy Documentation* [online]. [visitado 2026-03-08]. Disponible en: https://docs.flamapy.org/framework/transformations/fm_to_bdd (vid. pág. 29).
- THÜM, T.; KÄSTNER, C.; BENDUHN, F.; MEINICKE, J.; SAAKE, G. y LEICH, T., 2014. FeatureIDE: An extensible framework for feature-oriented software desarrollo. *Science of Computer Programming*. Vol. 79, págs. 70-85 (vid. págs. 11, 28).
- UNESCO, 2022. *Reimagining our futures together: A new social contract for education*. UNESCO (vid. pág. 1).
- UVL COMMUNITY, [s.f.]. *Universal Variability Language (UVL): Community-Driven Language for Variability Models* [online]. [visitado 2026-03-05]. Disponible en: <https://universal-variability-language.github.io/> (vid. págs. 14, 15).
- VEIGA MÉNDEZ, Antonio José, 2024. Norma ISO 9001 en Universidades: Un Enfoque para Mejorar la Gestión de la Calidad Académica. *UP•NIVERSITY Perspectivas*. Url: <https://es.linkedin.com/pulse/norma-iso-9001-en-universidades-un-enfoque-para-la-de-veiga-m%C3%A9ndez-q70uf>. [citation:3] (vid. pág. 1).
- WANG, Hongyu; ZHAO, Lu y HU, Jun, 2007. Formal Semantics and Verification for Feature Modeling. *Computer Software and Applications Conference (COMPSAC)*, págs. 148-155 (vid. pág. 8).

Apéndices

Proyectos y Avaes



REPORTE DEL PLANIFICADOR DE SECUENCIA DE ENSAMBLE

Ing. Maybel Díaz Capote
Universidad de las Ciencias Informáticas, La Habana, Cuba
Fecha: sábado 5 de octubre de 2013

DETALLES DEL REPORTE:

Caso de estudio: Bloque Corona
Reorientaciones: sí
Cambios de herramientas: no
Vértices: 17
Aristas: 136
Número total de hormigas: 11

Secuencias de ensamble:

Hormiga: 1
Reorientaciones: 2

Salida:
(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(6, +z)-(3, +z)-(2, +z)-(13, +z)-(1, +z)

Hormiga: 2
Reorientaciones: 3

Salida:
(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(4, +z)-(6, +z)-(3, +z)-(2, +z)-(13, +z)-(7, +y)-(17, +y)-(1, +y)

Hormiga: 3
Reorientaciones: 3

Salida:
(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(3, +z)-(6, +z)-(17, +z)-(1, +z)-(13, +z)-(7, +y)-(4, +y)-(2, +y)

Hormiga: 4
Reorientaciones: 2

Salida:
(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(13, +z)-(1, +z)-(6, +z)-(3, +z)-(2, +z)

Hormiga: 5
Reorientaciones: 3

Salida:
(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(12, +z)-(9, +z)-(5, +z)-(4, +z)-(6, +z)-(7, +z)-(17, +z)-(1, +z)-(2, +z)-(13, +z)-(8, +x)-(3, +x)

Hormiga: 6
Reorientaciones: 2

Salida:
(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(1, +z)-(6, +z)-(2, +z)-(13, +z)-(3, +z)

B.1. Otra sección de muestra

De una sola sentencia

B.2. Otro ensamble Industrial



REPORTE DEL PLANIFICADOR DE SECUENCIA DE ENSAMBLE

Ing. Maybel Díaz Capote
Universidad de las Ciencias Informáticas, La Habana, Cuba
Fecha: sábado 5 de octubre de 2013

DETALLES DEL REPORTE:

Caso de estudio: Bloque Corona
Reorientaciones: sí
Cambios de herramientas: no
Vértices: 17
Aristas: 136
Número total de hormigas: 11

Secuencias de ensamble:

Hormiga: 1
Reorientaciones: 2

Salida:
(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(6, +z)-(3, +z)-(2, +z)-(13, +z)-(1, +z)

Hormiga: 2
Reorientaciones: 3

Salida:
(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(4, +z)-(6, +z)-(3, +z)-(2, +z)-(13, +z)-(7, +z)-(17, +z)-(1, +z)

Hormiga: 3
Reorientaciones: 3

Salida:
(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(3, +z)-(6, +z)-(17, +z)-(1, +z)-(13, +z)-(7, +z)-(4, +z)-(2, +z)

Hormiga: 4
Reorientaciones: 2

Salida:
(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(13, +z)-(1, +z)-(6, +z)-(3, +z)-(2, +z)

Hormiga: 5
Reorientaciones: 3

Salida:
(11, +z)-(10, +z)-(16, +x)-(14, +x)-(15, +x)-(12, +z)-(9, +z)-(5, +z)-(4, +z)-(6, +z)-(7, +z)-(17, +z)-(1, +z)-(2, +z)-(13, +z)-(8, +x)-(3, +x)

Hormiga: 6
Reorientaciones: 2

Salida:
(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(1, +z)-(6, +z)-(2, +z)-(13, +z)-(3, +z)



REPORTE DEL PLANIFICADOR DE SECUENCIA DE ENSAMBLE

Ing. Maybel Díaz Capote
Universidad de las Ciencias Informáticas, La Habana, Cuba
Fecha: sábado 5 de octubre de 2013

DETALLES DEL REPORTE:

Caso de estudio: Bloque Corona
Reorientaciones: sí
Cambios de herramientas: no
Vértices: 17
Aristas: 136
Número total de hormigas: 11

Secuencias de ensamble:

Hormiga: 1

Reorientaciones: 2

Salida:

(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(17, +z)-(6, +z)-(3, +z)-(2, +z)-(13, +z)-(1, +z)

Hormiga: 2

Reorientaciones: 3

Salida:

(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(17, +z)-(6, +z)-(3, +z)-(2, +z)-(13, +z)-(7, -y)-(17, -y)-(1, -y)

Hormiga: 3

Reorientaciones: 3

Salida:

(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(3, +z)-(6, +z)-(17, +z)-(1, +z)-(13, +z)-(7, +y)-(4, +y)-(2, +y)

Hormiga: 4

Reorientaciones: 2

Salida:

(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(13, +z)-(1, +z)-(6, +z)-(3, +z)-(2, +z)

Hormiga: 5

Reorientaciones: 3

Salida:

(11, +z)-(10, +z)-(16, +x)-(14, +x)-(15, +x)-(12, +z)-(9, +z)-(5, +z)-(4, +z)-(6, +z)-(7, +z)-(17, +z)-(1, +z)-(2, +z)-(13, +z)-(8, +x)-(3, +x)

Hormiga: 6

Reorientaciones: 2

Salida:

(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(1, +z)-(6, +z)-(2, +z)-(13, +z)-(3, +z)

Hormiga: 7

Reorientaciones: 2

Salida:

(8, -x)-(14, -x)-(15, -x)-(16, -x)-(5, +z)-(4, +z)-(6, +z)-(7, +z)-(17, +z)-(2, +z)-(13, +z)-(1, +z)-(9, -z)-(10, -z)-(11, -z)-(12, -z)-(3, -z)

Hormiga: 8

Reorientaciones: 2

Salida:

(8, -x)-(14, -x)-(15, -x)-(16, -x)-(5, +z)-(4, +z)-(6, +z)-(7, +z)-(17, +z)-(2, +z)-(13, +z)-(1, +z)-(3, -z)-(9, -z)-(10, -z)-(11, -z)

Hormiga: 9

Reorientaciones: 3

Salida:

(8, -x)-(14, -x)-(15, -x)-(16, -x)-(10, -z)-(11, -z)-(9, +z)-(5, +z)-(4, +z)-(7, +z)-(17, +z)-(2, +z)-(13, +z)-(1, +z)-(6, +z)-(3, +z)-(12, -x)

Hormiga: 10

Reorientaciones: 2

Salida:

(8, -x)-(14, -x)-(15, -x)-(16, -x)-(5, +z)-(4, +z)-(6, +z)-(7, +z)-(17, +z)-(2, +z)-(13, +z)-(1, +z)-(3, -z)-(9, -z)-(10, -z)-(11, -z)-(12, -z)

Hormiga: 11

Reorientaciones: 2

Salida:

(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(9, -z)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(1, +z)-(6, +z)-(3, +z)-(2, +z)-(13, +z)

Secuencia de salida premiada

Reorientaciones: 2

Salida:

(8, -x)-(15, -x)-(14, -x)-(16, -x)-(10, -z)-(11, -z)-(12, +z)-(9, +z)-(5, +z)-(4, +z)-(7, +z)-(6, +z)-(13, +z)-(3, +z)-(17, +z)-(2, +z)-(1, +z)